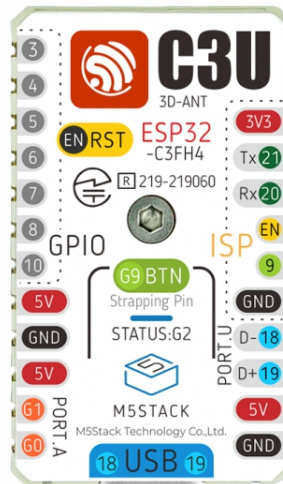


C3U-Metal

Programming the ESP32-C3 from the bare metal up

A guided tour of a complete program for the M5Stack M5Stamp C3U: the start-up code, the drivers, the C library and an interactive Lua console, with no operating system underneath.



The M5Stamp C3U. Picture: M5Stack, from their documentation.

Contents

1	What C3U-Metal is	5	7	SystemInit(): watchdogs, clock and cycle counter	28
1.1	What it does	5	7.1	Why a separate SystemInit()?	28
1.2	Bare metal	5	7.2	Stopping the watchdogs	28
1.3	The layers of the project	6	7.3	The CPU clock	29
1.4	What you need	6	7.4	The cycle counter	29
1.5	Setting up on Windows, macOS and Linux	6	7.5	Time since start-up	30
1.6	Your first build	7			
2	The board and the chip	8	8	Registers and the GPIO driver	31
2.1	The M5Stamp C3U	8	8.1	From a register bit to a pin	31
2.2	Numbers, bits and hexadecimal	8	8.2	The registers	31
2.3	The ESP32-C3 and RISC-V	9	8.3	Checking a pin number	32
2.4	The CPU's own registers	9	8.4	Configuring the pad	32
2.5	Memory: ROM, flash and RAM	9	8.5	Outputs	32
2.6	Hardware registers and memory-mapped I/O	10	8.6	Inputs and pull resistors	33
2.7	Control and status registers	11	8.7	Writing and reading	33
2.8	The clock and the CPU's speed	11			
3	Building and flashing	12	9	The SK6812 RGB LED	34
3.1	Cross-compiling	12	9.1	One wire, 24 bits	34
3.2	The toolchain file and the preset	12	9.2	Timing in CPU cycles	35
3.3	CMakeLists.txt, part by part	13	9.3	Why assembly, and why RAM	35
3.4	Everyday commands	14	9.4	Walking through sk6812_send_grb	35
3.5	The Python tools	14	9.5	Calibrating the timing	36
3.6	VS Code	15			
4	How it boots	16	10	The USB console driver	38
4.1	After a reset, the ROM runs	16	10.1	USB built into the chip	38
4.2	First check: download mode	16	10.2	Three registers	38
4.3	Second check: the direct-boot magic number	16	10.3	Sending bytes	38
4.4	How ESP-IDF boots instead	17	10.4	Waiting for room, but not for ever	39
4.5	The whole boot, step by step	17	10.5	Ending a transfer	39
4.6	What the ROM leaves behind	17	10.6	Receiving bytes	40
4.7	Watching the boot with a debugger	18	10.7	The whole path	40
			10.8	DTR and RTS	40
5	The linker script and the memory map	19	11	newlib and syscalls.c	41
5.1	Sections: sorting a program into kinds of memory	19	11.1	What a C library needs	41
5.2	Where it runs and where it is stored	19	11.2	How printf reaches the screen	41
5.3	The MEMORY block	19	11.3	Reading keys: _read()	42
5.4	Code and constants in flash	20	11.4	Where malloc gets memory: _sbrk()	42
5.5	Code that runs from RAM	20	11.5	Only one task: _getreent()	42
5.6	Variables: .data and .bss	21	11.6	Things that don't exist here	43
5.7	The global pointer	22	11.7	Time	43
5.8	Stack and heap	22			
5.9	The whole picture	22	12	Lua and the REPL	44
5.10	Linker symbols in C	23	12.1	Lua on the board	44
6	crt0.S, the C runtime	24	12.2	Starting Lua: main.c	44
6.1	What C needs before main()	24	12.3	The Lua C API in five ideas	44
6.2	Reading RISC-V assembly	24	12.4	The REPL at a glance	45
6.3	The plan	24	12.5	Reading a line	45
6.4	Step 1: the stack pointer and the global pointer	25	12.6	Expression or statement?	46
6.5	Step 2: painting the stack	25	12.7	Running a chunk, and Ctrl-C	47
6.6	Step 3: where traps go	26	12.8	Printing results and the main loop	48
6.7	Step 4: SystemInit()	26			
6.8	Step 5: copying and clearing memory	26	13	The board's Lua functions	49
6.9	Steps 6 and 7: start-up functions and main()	27	13.1	A C function that Lua can call	49
			13.2	Checking arguments: led()	49
			13.3	delay() and millis()	49
			13.4	The gpio table	50
			13.5	help(): everything the board adds	51
			13.6	mem(): the heap and the stack	51

13.7	peek() and hex()	52	15.4	Step 3: the CMSIS-style split	61
13.8	crash()	52	15.5	Step 4: newlib, the USB console and code in RAM	61
13.9	Adding a function of your own	53	15.6	Step 5: Lua	61
14	When things go wrong	54	15.7	After step 5	61
14.1	Traps: exceptions and interrupts	54	16	Exercises	62
14.2	The trap vector table	54	16.1	The exercises	62
14.3	The trap handler	54	16.2	Working on an exercise	62
14.4	The crash report	55	16.3	What each exercise asks	62
14.5	Reading a report	56	16.4	Worked solutions and SOLUTION builds	63
14.6	Looking inside a running board	56	Appendix A	Register quick reference	64
14.7	Debugging in VS Code	57	A.1	Peripheral registers	64
14.8	When the stack overflows	58	A.2	CSRs	64
14.9	When nothing appears, or the board is stuck	58	Appendix B	Glossary	65
15	The lessons in the git history	59	Appendix C	Troubleshooting	67
15.1	Exploring the history	59			
15.2	Step 1: assembly only	59			
15.3	Step 2: a C runtime	60			

About this manual

C3U-Metal is a program for the ESP32-C3 chip on the M5Stamp C3U board. It does everything itself, from the first instruction after power-on to a prompt where you can type Lua, and it is small enough to read from start to finish. This manual goes through it in the order the board runs it.

Who it is for

It is written for members of a school engineering club: students and their teacher. You should have written a little code before, in C or Python for example. You don't need to know anything about electronics, assembly language or linkers. Each idea is explained the first time it appears, and Appendix B is a glossary you can come back to.

How it is organised

Chapters	What they cover
1 to 3	What the project is, the board and the chip, and how to build the program and put it on the board.
4 to 7	Everything that happens between power-on and <code>main()</code> : the chip's ROM, the linker script, the C runtime and chip set-up.
8 to 11	The drivers and the C library: pins, the RGB LED, the USB console and newlib.
12 and 13	Lua, the interactive console, and the functions the board adds to Lua.
14	Crashes, debugging, and getting a stuck board back.
15 and 16	The lessons stored in the git history, and the exercises.
Appendices	A register reference, a glossary and a troubleshooting table.

Conventions

- **Code excerpts are copied exactly from the repository.** The bar above each one names the file, the function or part it comes from, and its line numbers, so you can open the file and find it. Comments are shown in green. Long lines wrap onto the next row.
- File paths such as `boot/crt0.S` are relative to the `C3U-Metal` folder.
- Green boxes headed *Run in a terminal* hold commands for the ESP-IDF terminal on your computer. Boxes headed *Type at the console* hold Lua to type at the board's `>` prompt.
- Numbers starting with `0x` are hexadecimal. Section 2.2 explains them.
- Notes in blue explain *why* something is done. Green notes suggest something to try. Amber notes warn about a mistake that is easy to make.

The manual describes the `main` branch as it was in September 2026, including the eight exercises and their worked solutions. If a file has changed since, trust the file.

For teachers: one way to run it

Session	Read	Do
1	Chapters 1 to 3	Install the tools, build and flash, play at the console, run the self-test.
2	Chapters 4 to 6	Find the sections in the map file. Follow the boot in the debugger.
3	Chapters 7 to 9	Drive an LED on a breadboard with the <code>gpio</code> functions. Measure the RGB LED's timing.
4	Chapters 10 to 13	Trace <code>print()</code> down to the USB registers. Add a Lua function of your own.
5	Chapter 14	Crash the board on purpose and read the report.
6 onwards	Chapters 15 and 16	Walk through the lessons in the history, then start the exercises.

What C3U-Metal is

C3U-Metal is a program for the ESP32-C3 chip that does everything itself. There is no bootloader, no operating system and no framework. The chip's built-in start-up code jumps straight into our code, and every line that runs after that is in the repository.

1.1 What it does

Plug the board into a computer and open its console, and you get a prompt where you can type *Lua*, a small programming language. Each line runs on the board as soon as you press Enter. These examples come from the project's `README.md`:

README.md	examples at the Lua console	lines 18–23
18	> led(0, 0, 40)	-- the RGB LED turns blue
19	> for i = 1, 5 do led(40, 0, 0) delay(200) led(0, 0, 0) delay(200) end	
20	> gpio.output(4) gpio.write(4, 1)	-- GPIO4 goes high
21	> gpio.output(8) while true do gpio.write(8, button()) delay(10) end	-- GPIO8 follows the button; Ctrl-C stops it
22	> mem()	-- how the RAM is being used
23	> help()	-- everything the board adds to Lua

Text after `--` is a Lua comment. The functions `led()`, `delay()`, `gpio.output()`, `mem()` and `help()` are not part of Lua. C3U-Metal adds them, and this manual shows how, all the way down to the bits written into the chip.

1.2 Bare metal

Most programs run on top of other software. A program on a laptop runs on an operating system, which loads it, gives it memory, and handles the screen, the keyboard and the files for it. Programs for small chips usually have helpers too. Code that runs with nothing at all underneath it is called **bare metal**, and that is where C3U-Metal gets its name.

Here is how it compares with the two usual ways of programming an ESP32-C3:

	Arduino	ESP-IDF	C3U-Metal
What you write	<code>setup()</code> and <code>loop()</code> , using Arduino libraries	a program that uses ESP-IDF's libraries	everything, from the first instruction
How your code is started	built on ESP-IDF, so it starts the same way	the chip's ROM loads a <i>bootloader</i> from flash; the bootloader loads the program and starts FreeRTOS	the ROM jumps straight to <code>_start</code> in <code>boot/crt0.S</code>
Operating system	FreeRTOS, hidden	FreeRTOS	none
Drivers	Arduino and ESP-IDF libraries	ESP-IDF components	our own, in <code>drivers/</code>
Good for	getting something working quickly	large programs that need many features and several tasks at once	seeing how all of it works

FreeRTOS is a small real-time operating system. It lets a program run several *tasks* that take turns on the CPU, and gives them timers and ways to pass messages to each other. A **bootloader** is a small program whose only job is to get the real program into memory and start it.

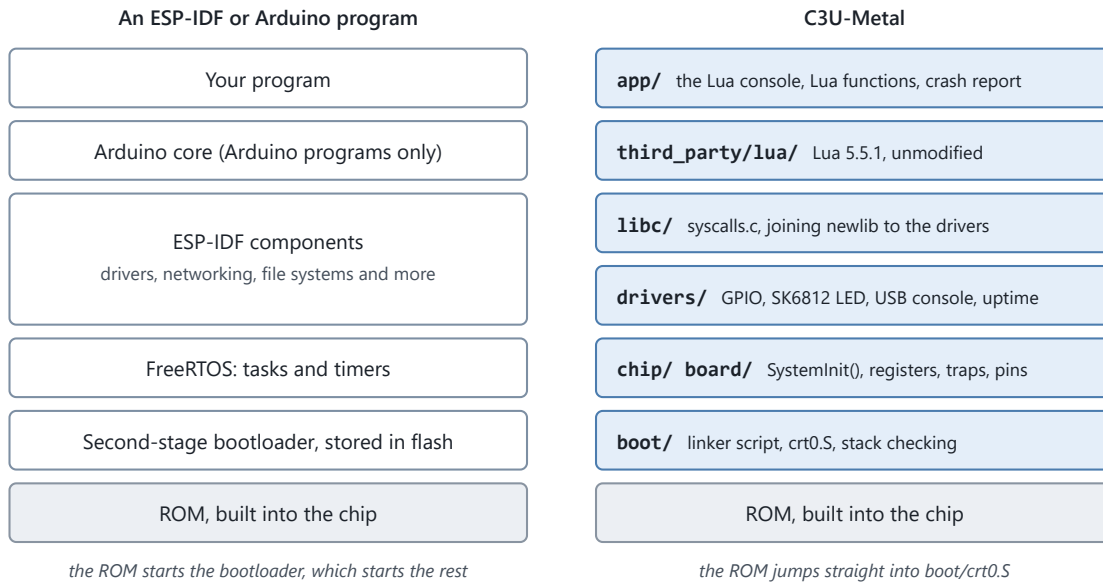
So why do without all of that? Because it hides the interesting parts. With C3U-Metal you can see how the chip starts, where each part of the program lives in memory, what a driver writes to the hardware, how `printf` gets its text to your screen, and what happens in a crash. The price is that C3U-Metal does far less than ESP-IDF: it runs no tasks, handles no interrupts and has no networking.

Only the tools come from ESP-IDF

C3U-Metal uses the compiler, CMake, Python and the debugging tools that are installed with ESP-IDF. None of ESP-IDF's own code ends up on the board.

1.3 The layers of the project

The source is sorted into folders, one for each layer. Lower layers know nothing about the layers above them.



Left: what sits between the chip and your code in a usual program. Right: C3U-Metal's folders. Everything in blue is in the repository.

Three more folders don't end up on the board: `tools/` holds Python scripts that flash, monitor and test the board; `docs/` holds diagrams, this manual and the exercises; `solutions/` holds worked solutions to the exercises.

1.4 What you need

Hardware

- **An M5Stack M5Stamp C3U** (M5Stack product code C122-B). M5Stack's documentation and pin-out are at docs.m5stack.com/en/core/stamp_c3u. The *M5Stamp C3U Mate with Pin Headers* (K122) comes with headers for a breadboard.
- **A USB-C cable that carries data.** A charge-only cable powers the board, but the board never shows up as a serial port.
- Optional, for the GPIO exercises: a breadboard, jumper wires, an LED and a 330 Ω resistor.

Software

Everything comes with **ESP-IDF v6.0.1**, Espressif's development framework: the RISC-V GCC compiler, CMake and Ninja, Python with the pyserial and esptool packages, and OpenOCD and GDB for debugging. Other ESP-IDF versions will probably work, but only v6.0.1 has been tested.

1.5 Setting up on Windows, macOS and Linux

- 1 **Install ESP-IDF** by following Espressif's *Get Started* guide for the ESP32-C3. It has instructions for Windows, macOS and Linux.
- 2 **Open a terminal with ESP-IDF's tools loaded.** On Windows, use the *ESP-IDF PowerShell* shortcut the installer creates. On macOS and Linux, run the activation command the installer gives you, for example `. $HOME/esp/esp-`

idf/export.sh.

3 Check that the tools are there:

Run in a terminal

```
riscv32-esp-elf-gcc --version
cmake --version
python -c "import serial, esptool; print('ok')"
```

- 4 Linux only:** allow your user to open the serial port with `sudo usermod -aG dialout $USER`, then log out and back in. For debugging, also install OpenOCD's USB rules: `sudo cp $OPENOCD_SCRIPTS/../../contrib/60-openocd.rules /etc/udev/rules.d/`
- 5 Windows, and only for debugging:** if OpenOCD can't find the board, use Zadig (zadig.akeo.ie) to install the WinUSB driver on the *USB JTAG/serial debug unit* interface. The serial port itself needs no driver.

Always use the ESP-IDF terminal

Run every command in this manual in that terminal, in the `C3U-Metal` folder. In an ordinary terminal the compiler and Python packages won't be found. If you use VS Code, start it from the ESP-IDF terminal with `code .` for the same reason.

1.6 Your first build

Run in a terminal

```
cmake --preset default          # once: sets up the build folder
cmake --build build -t flash    # compile, link and write it to the board
python tools/monitor.py         # the Lua console; Ctrl-J quits
```

The first build also compiles Lua, so it takes a little longer. After that, only what changed is rebuilt. In the monitor, press Enter for a `>` prompt and type `help()`.

Only one program can use the serial port at a time, so close the monitor before you flash again. Any serial terminal works instead of `monitor.py`, for example PuTTY, MobaXterm or `screen`, and the speed setting doesn't matter over USB. Chapter 3 explains each of these commands.

The board and the chip

Before looking at the code, it helps to know what the code talks to: a small board, one chip with a RISC-V CPU inside, a few kinds of memory, and a large number of hardware registers.

2.1 The M5Stamp C3U

The M5Stamp C3U is a small board built around Espressif's ESP32-C3 chip. C3U-Metal uses only a few of its parts. The picture on the cover labels pins G2, G9 and so on; in the code they are GPIO2, GPIO9. **GPIO** means *general-purpose input/output*: a pin the program can switch high or low, or read.

Part	Pin	Used for
RGB LED (SK6812)	GPIO2	<code>led()</code> , chapter 9
Button	GPIO9	<code>button()</code> ; also download mode (4.2)
USB-C socket	GPIO18, GPIO19	the console and the debugger, chapter 10
Edge pins	for example GPIO4	your own circuits, chapter 8

The pins are defined once, in `board/board.h`, so no other file has to repeat the numbers:

<code>board/board.h</code> the pins C3U-Metal uses		lines 19–22
19	<code>/* Pins */</code>	
20	<code>#define LED_PIN 2</code>	<code>/* data line of the SK6812 RGB LED */</code>
21	<code>#define BTN_PIN 9</code>	<code>/* the button pulls the pin to ground: reads 0 while pressed.</code>
22		<code>Held while plugging in, it starts the chip's download mode. */</code>

Pulling a pin *to ground* means connecting it to 0 volts, so the pin reads as 0. Chapter 8 explains why the button reads 1 when it is not pressed.

2.2 Numbers, bits and hexadecimal

Hardware code is full of numbers such as `0x6000403C`. The `0x` means the number is written in **hexadecimal**, base 16, with the digits 0 to 9 and A to F. Hexadecimal is used because each digit stands for exactly four bits, so you can see which bits are set: `0x200` is `0010 0000 0000` in binary, which is bit 9 on its own.

A **bit** is a single 0 or 1. The bits of a number are counted from 0, starting at the right, so a 32-bit value has bits 0 to 31. Eight bits make a **byte**, and on this chip 32 bits (4 bytes) make a **word**. A group of bits with one meaning is called a **field** and is written with its highest and lowest bit: *bits 11:10* means bits 11 and 10 taken together.

C, Lua and assembly all use the same operators to work with bits:

Written	Means	Example
<code>1u << 9</code>	a number with only bit 9 set (<code>u</code> makes it unsigned)	<code>0x00000200</code>
<code>a b</code>	bits that are set in <code>a</code> , in <code>b</code> , or in both	set a bit: <code>x (1u << 9)</code>
<code>a & b</code>	bits that are set in both	test a bit: <code>x & (1u << 9)</code>
<code>~a</code>	every bit flipped	<code>~0xFFFFu</code> is <code>0xFFFFF000</code>
<code>x >> 9</code>	shift right: bit 9 moves down to bit 0	<code>(x >> 9) & 1u</code> is bit 9 of <code>x</code>

2.3 The ESP32-C3 and RISC-V

A CPU understands a fixed set of *instructions*, such as "add these two numbers" or "jump to this address". The CPU in the ESP32-C3 uses RISC-V, an open instruction set that anyone may use. RISC-V comes in pieces, and `CMakeLists.txt` tells the compiler exactly which pieces this chip has:

```
CMakeLists.txt the instruction set lines 23–26
23 # The ESP32-C3's instruction set: RV32IMC plus the CSR and FENCE.I instructions.
24 # It must match a variant of the toolchain's newlib and libgcc, and is used for
25 # compiling and for linking.
26 set(ARCH_FLAGS -march=rv32imc_zicsr_zifencei -mabi=ilp32)
```

- `rv32`: 32-bit RISC-V. Addresses and the CPU's registers are 32 bits wide.
- `i`: the basic integer instructions. `m`: multiply and divide.
- `c`: *compressed* instructions. Common instructions take 16 bits instead of 32, which makes the program smaller.
- `zicsr`: the instructions that read and write control and status registers (section 2.7). `zifencei`: the FENCE.I instruction.
- `-mabi=ilp32`: in C, `int`, `long` and pointers are all 32 bits.

There is no `f`, so the chip has no floating-point instructions. The compiler turns maths with `float` and `double` into calls to helper functions in `libgcc`, one of the libraries linked into the program.

2.4 The CPU's own registers

A **register** is a small, very fast piece of storage. The CPU has 32 of them for doing its work, plus the program counter. RISC-V gives them numbers, but code uses their *names*, which describe their jobs:

Name	Job	Where you meet it
<code>pc</code>	program counter: the address of the instruction about to run	<code>read_registers.py</code>
<code>sp</code>	stack pointer: the top of the stack (chapter 6)	<code>crt0.S</code> step 1
<code>ra</code>	return address: where a function goes back to when it returns	the crash report
<code>gp</code>	global pointer: a fixed address near the small global variables (5.7)	<code>crt0.S</code> , the linker script
<code>a0</code> to <code>a7</code>	arguments passed to a function; <code>a0</code> also carries its result	<code>sk6812.S</code> , <code>trap.S</code>
<code>t0</code> to <code>t6</code>	temporary values that a called function may overwrite	the loops in <code>crt0.S</code>
<code>s0</code> to <code>s11</code>	saved values that a function must put back before it returns	step 1's <code>main.S</code>
<code>zero</code>	always 0; writing to it does nothing	<code>sw zero, 0(t1)</code> in <code>crt0.S</code>

The rules about which register carries what, and which ones a function may change, are the **calling convention**. They are what let C call assembly and assembly call C. `drivers/sk6812.S`, for example, promises that it "uses only `a0`-`a4` and `t0`-`t5`, which the calling convention lets it overwrite", so C code can call it safely.

2.5 Memory: ROM, flash and RAM

The chip uses three kinds of memory:

- **ROM** (read-only memory) holds a program Espressif built into the chip, at address `0x40000000`. It can't be changed. It runs first after every reset (chapter 4).
- **Flash** is where our program is stored. It keeps its contents without power and can be rewritten by `esptool`. In C3U-Metal's start-up mode the first 4 MB of flash appear twice: at `0x42000000` for running code and at `0x3C000000` for reading data.
- **RAM** is fast memory that can be written at any time but forgets everything when the power goes. The ESP32-C3 has 400 KB of it. The ROM keeps the top part for itself, from `0x3FCD0000` up, so C3U-Metal uses the 320 KB below that, from `0x3FC80000`. The same RAM also appears at `0x40380000`, and code in RAM has to be run from that address.

The CPU sees all of these, and the hardware registers too, as one long row of numbered bytes from `0x00000000` to `0xFFFFFFFF`. This is the **address space**, and each part of the chip is given a range in it:

Address	What is there	What C3U-Metal uses it for
<code>0x3C000000</code>	DROM: flash, read as data a view of the first 4 MB of flash	constants and strings (.rodata), and the stored copies of what goes in RAM
<code>0x3FC80000</code>	RAM for C3U-Metal (320 KB) up to <code>0x3FCD0000</code> ; the data address	global variables, the heap and the stack; code for RAM is copied in through here
<code>0x3FCD0000</code>	RAM the ROM keeps for itself the top part of the RAM	not used by C3U-Metal
<code>0x40000000</code>	ROM code built into the chip	runs first after every reset
<code>0x40380000</code>	The same RAM, code address <code>0x700000</code> above the data address	<code>sk6812_send_grb</code> runs here, because the CPU runs RAM code only from this address
<code>0x42000000</code>	IROM: the same flash, run as code a view of the first 4 MB of flash	<code>_start</code> at <code>0x42000008</code> , and all the other code
<code>0x6000xxxx</code>	Peripheral registers memory-mapped I/O (section 2.6)	GPIO <code>0x60004000</code> , RTC_CNTL <code>0x60008000</code> , IO_MUX <code>0x60009000</code> , TIMG0 <code>0x6001F000</code> , USB <code>0x60043000</code> , SYSTEM <code>0x600C0000</code>

The parts of the address space C3U-Metal uses. Addresses increase down the page. Sizes are not to scale.

2.6 Hardware registers and memory-mapped I/O

A **peripheral** is a part of the chip that isn't the CPU: the GPIO pins, the USB port, timers, watchdogs. Each is controlled through its own **hardware registers**. Writing a value to a register changes what the hardware does, and reading one tells you what the hardware is doing.

On the ESP32-C3, hardware registers have addresses, just like memory. To switch a pin on, the program writes a number to an address. This is called **memory-mapped I/O** (I/O means input and output). C3U-Metal's C code reaches every register through one macro:

```
chip/esp32c3-regs.h REG() lines 20–23
20 #ifndef __ASSEMBLER__
21 #include <stdint.h>
22 #define REG(addr) (*(volatile uint32_t *) (addr)) /* read or write a 32-bit hardware register */
23 #endif
```

- `(addr)` is a plain number, such as `0x6000403C`.
- `(volatile uint32_t *)` tells C to treat that number as the address of a 32-bit unsigned value.
- The `*` in front reads or writes the value at that address. So `REG(GPIO_IN)` reads the GPIO_IN register, and `REG(GPIO_OUT_W1TS) = 1u << pin;` writes one.
- `volatile` warns the compiler that the value can change by itself and that every read and write matters. Without it, the compiler might decide that writing the same register twice is pointless and remove one of the writes.

The `#ifndef __ASSEMBLER__` around it is there because the same header is also included by assembly files, which don't understand C.

Try this

At the console, type `hex(peek(0x6000403C))`. That reads GPIO_IN, which has one bit for each pin. Hold the button and type it again: bit 9 changes, so the value changes by `0x200`.

2.7 Control and status registers

Some registers belong to the CPU itself and have no memory address. RISC-V calls them **control and status registers**, or **CSRs**. Each has a number instead, and two special instructions use them: `csrr` reads a CSR into an ordinary register, and `csrw` writes one. C has no way to produce those instructions, which is why a few small functions are written in assembly.

C3U-Metal uses the standard trap CSRs, `mtvec`, `mcause`, `mepc`, `mtval` and `mscratch`, which chapter 14 explains. It also uses three CSRs that Espressif added to count CPU cycles, numbered `0x7E0` to `0x7E2` ([section 7.4](#)).

2.8 The clock and the CPU's speed

A CPU does its work in steps timed by a **clock** signal. The board has a 40 MHz crystal, which makes 40 million clock cycles a second. After a reset the chip divides that by 2 and runs the CPU at 20 MHz. `SystemInit()` switches it to the full 40 MHz (chapter 7), and `board.h` records that speed for the rest of the code:

board/board.h CPU speed

lines 9–17

```
9  /*
10 * CPU speed. SystemInit() (chip/system_esp32c3.c) runs the CPU straight from
11 * the board's 40 MHz crystal, so the cycle counter counts CPU_MHZ million
12 * cycles a second. Delays, uptime and the LED's pulses are all timed with it.
13 * Files that only work at 40 MHz stop the build with #error if this changes.
14 */
15 #define CPU_MHZ          40
16 #define CPU_CYCLES_PER_MS (CPU_MHZ * 1000)
17 #define CPU_CYCLES_PER_US CPU_MHZ
```

At 40 MHz one cycle lasts 25 nanoseconds (a nanosecond is a billionth of a second). The cycle counter counts these cycles, so C3U-Metal uses it as its clock: 40 000 cycles make a millisecond and 40 make a microsecond. That is exactly what `CPU_CYCLES_PER_MS` and `CPU_CYCLES_PER_US` say.

Building and flashing

Your computer turns the source files into a program for a different CPU, then writes that program into the board's flash. This chapter covers the build files and the Python tools that talk to the board.

3.1 Cross-compiling

Your computer's CPU is not a RISC-V CPU, so its ordinary compiler can't make programs for the board. A **cross-compiler** runs on one kind of computer and produces code for another. Espressif's is `riscv32-esp-elf-gcc`, installed with ESP-IDF. Building has three stages:

1. **Compile**: each `.c` file becomes an *object file*, machine code sorted into named sections (chapter 5).
2. **Assemble**: each `.S` file goes through the C preprocessor, so `#include` and `#define` work, and then becomes an object file too.
3. **Link**: the *linker* joins the object files and libraries into one program and gives every function and variable an address, following the *linker script* `boot/c3u-metal.ld`.

The results go in the `build` folder. `c3u-metal.elf` is the program with its addresses, names and debugging information, for the debugger. `c3u-metal.bin` holds only the bytes that go into flash. `c3u-metal.map` reports where every section, function and variable went. **CMake** reads `CMakeLists.txt` and writes instructions for **Ninja**, which runs the compiler and only rebuilds what has changed.

3.2 The toolchain file and the preset

CMake normally builds for the computer it runs on. A *toolchain file* tells it to build for somewhere else:

cmake/riscv32-esp-elf.cmake the toolchain file, after its header lines 6–19

```

6 set(CMAKE_SYSTEM_NAME Generic)           # "Generic": no operating system on the target
7 set(CMAKE_SYSTEM_PROCESSOR riscv32)
8
9 set(CMAKE_C_COMPILER riscv32-esp-elf-gcc)
10 set(CMAKE_ASM_COMPILER riscv32-esp-elf-gcc) # gcc runs the C preprocessor on .S files, then assembles them
11
12 # CMake checks the compiler by building a small test program. A normal program
13 # can't link without our linker script and crt0.S, so build a library instead.
14 set(CMAKE_TRY_COMPILE_TARGET_TYPE STATIC_LIBRARY)
15
16 # Programs (such as Python) come from this PC; libraries and headers only from the toolchain.
17 set(CMAKE_FIND_ROOT_PATH_MODE_PROGRAM NEVER)
18 set(CMAKE_FIND_ROOT_PATH_MODE_LIBRARY ONLY)
19 set(CMAKE_FIND_ROOT_PATH_MODE_INCLUDE ONLY)

```

- `Generic` means there is no operating system on the target. The same `gcc` compiles C and assembles `.S` files.
- CMake tests a new compiler by building a tiny program. That program couldn't link without our linker script and `crt0.S`, so CMake is told to build a library for the test instead.
- The last three lines make CMake look for programs, such as Python, on your computer, but for libraries and headers only among the cross-compiler's own files.

`CMakePresets.json` saves typing. `cmake --preset default` means: use Ninja, put the build in `build`, use this toolchain file, and write `compile_commands.json` so that VS Code knows each file's compiler options.

3.3 CMakeLists.txt, part by part

The file starts by choosing the toolchain file and declaring a project with C and assembly (ASM) files. `ARCH_FLAGS` is the instruction set from [section 2.3](#). It is used for linking as well as compiling, because the linker picks the matching version of the C library. Then comes Lua:

CMakeLists.txt Lua as a static library

lines 42–47

```
42 add_library(lua STATIC ${LUA_FILES})
43 target_compile_options(lua PRIVATE ${ARCH_FLAGS} -O2 -g -Wall -Wextra)
44 target_include_directories(lua PUBLIC ${LUA_DIR})
45 # 32-bit integers and floats. PUBLIC, because our C code must use the same
46 # setting, or it and Lua would disagree about what a Lua number is.
47 target_compile_definitions(lua PUBLIC LUA_32BITS)
```

Lua is built once as a *static library*, a bundle of object files the linker takes pieces from. The list above these lines names the files: the core of Lua, and the standard libraries that make sense without files or an operating system, so no `io`, `os`, `package` or `debug`. `LUA_32BITS` is explained in [section 12.1](#).

CMakeLists.txt the program's source files

lines 52–69

```
52 set(SOURCES
53     boot/crt0.S           # the C runtime: the chip starts running here
54     boot/stack_check.c
55     chip/system_esp32c3.c # chip setup
56     chip/cpu.S
57     chip/trap.S
58     drivers/gpio.c
59     drivers/sk6812.c
60     drivers/sk6812.S
61     drivers/usb_serial.c
62     drivers/uptime.c
63     libc/syscalls.c       # what newlib needs from an "operating system"
64     app/main.c
65     app/repl.c
66     app/lua_hw.c
67     app/lua_sys.c
68     app/trap_report.c)
69 set(INCLUDE_DIRS boot chip board drivers libc app)
```

That list is the whole of C3U-Metal's own code. The lines after it let the build take some files from a worked solution instead ([section 16.4](#)); in a normal build they change nothing.

CMakeLists.txt compiling and linking

lines 115–132

```
115 add_executable(c3u-metal ${SOURCE_PATHS})
116 set_target_properties(c3u-metal PROPERTIES SUFFIX .elf)
117
118 target_include_directories(c3u-metal PRIVATE ${INCLUDE_PATHS})
119 target_compile_options(c3u-metal PRIVATE
120     ${ARCH_FLAGS} -g -Og -Wall -Wextra
121     -fno-asynchronous-unwind-tables -fno-unwind-tables) # no C++ exceptions, so no unwind tables
122
123 pick_path(boot/c3u-metal.ld LINKER_SCRIPT)
124 target_link_options(c3u-metal PRIVATE
125     ${ARCH_FLAGS}
126     -nostdlib -nostartfiles           # our own crt0.S; the libraries are listed below
127     -T${LINKER_SCRIPT}
128     -Wl,-Map=${CMAKE_CURRENT_BINARY_DIR}/c3u-metal.map)
129 set_target_properties(c3u-metal PROPERTIES LINK_DEPENDS ${LINKER_SCRIPT}) # relink when it changes
130 target_link_libraries(c3u-metal PRIVATE
131     lua
132     -Wl,--start-group c m gcc -Wl,--end-group) # newlib, its maths library, and compiler helpers
```

- `-g` keeps debugging information, `-Og` optimises without confusing the debugger, and `-Wall -Wextra` turn on most warnings.
- `-nostdlib -nostartfiles` stop the compiler adding its own start-up code and default libraries. C3U-Metal has its own start-up code, `crt0.S`, and names the libraries it wants.
- `-T` gives the linker script, and `-Map` asks for the map file.

- The libraries are Lua, then `c` (newlib, the C library), `m` (its maths functions) and `gcc` (compiler helpers). The group lets them use each other's functions in any order.

CMakeLists.txt making the flash image lines 134–143

```

134 # The raw flash image. It starts with the direct-boot magic number from the linker script.
135 get_filename_component(TOOLCHAIN_BIN ${CMAKE_C_COMPILER} DIRECTORY)
136 find_program(RISCV_OBJCOPY riscv32-esp-elf-objcopy HINTS ${TOOLCHAIN_BIN} REQUIRED)
137 find_program(RISCV_SIZE riscv32-esp-elf-size HINTS ${TOOLCHAIN_BIN} REQUIRED)
138 set(IMAGE ${CMAKE_CURRENT_BINARY_DIR}/c3u-metal.bin)
139 add_custom_command(TARGET c3u-metal POST_BUILD
140     COMMAND ${RISCV_OBJCOPY} -O binary ${TARGET_FILE:c3u-metal} ${IMAGE}
141     COMMAND ${RISCV_SIZE} ${TARGET_FILE:c3u-metal}
142     BYPRODUCTS ${IMAGE}
143     VERBATIM)

```

`objcopy -O binary` keeps only the bytes that belong in flash, and `size` prints how big each part of the program is. The end of the file adds *targets* that run the Python tools. You name a target with `-t`: `flash` builds first, and `selftest` flashes first.

3.4 Everyday commands

Command	What it does
<code>cmake --preset default</code>	Once, and after changing <code>CMakeLists.txt</code> : set up the build folder.
<code>cmake --build build</code>	Compile and link: <code>c3u-metal.elf</code> , <code>.bin</code> and <code>.map</code> .
<code>cmake --build build -t flash</code>	Build, then write the program to the board.
<code>python tools/monitor.py</code>	The Lua console; <code>Ctrl + J</code> quits. Also <code>-t monitor</code> .
<code>python tools/selftest.py</code>	Type 15 checks into the console and report PASS or FAIL.
<code>cmake --build build -t selftest</code>	Flash, then run the self-test.
<code>python tools/read_registers.py</code>	Pause the CPU for a moment over JTAG and show its registers (section 14.6).

Moved or copied the folder?

The build folder remembers full paths. If you get build errors after moving or copying the project, delete the `build` folder and run `cmake --preset default` again.

3.5 The Python tools

The tools share `tools/board.py`, which finds the board by its USB IDs. Every USB device reports a *vendor ID* (who made it) and a *product ID*. The ESP32-C3's built-in USB port reports Espressif's IDs whatever program is running, so the tools find the board even when the program on it is broken:

tools/board.py `find_port()` lines 17–29

```

17 ESPRESSIF_VID = 0x303A
18 USB_SERIAL_JTAG_PID = 0x1001
19
20 NOT_FOUND = ("M5Stamp C3U not found. Plug it in. If it still isn't found, unplug it, "
21             "hold its button while plugging it back in, and try again.")
22
23
24 def find_port():
25     """The board's serial port name (such as COM11 or /dev/ttyACM0), or None."""
26     for port in sorted(list_ports.comports(), key=lambda p: p.device):
27         if port.vid == ESPRESSIF_VID and port.pid == USB_SERIAL_JTAG_PID:
28             return port.device
29     return None

```

When `board.py` opens the port, it first switches off the DTR and RTS lines, because a program that changes them can reset the board ([section 10.8](#)). If the port is busy, it tries again for a few seconds.

flash.py

tools/flash.py main(): running esptool

lines 47–55

```
47 # esptool is a Python package, so run it with this same Python. esptool 5 (ESP-IDF 6)
48 # spells its commands with dashes; esptool 4 (ESP-IDF 5) only knows the underscores.
49 dashes = int(esptool.__version__.split(".")[0]) >= 5
50 run = [sys.executable, "-m", "esptool", "--chip", "esp32c3", "--port", port]
51 if args.erase:
52     result = subprocess.run(run + ["erase-flash" if dashes else "erase_flash"])
53     if result.returncode != 0:
54         sys.exit(result.returncode)
55 result = subprocess.run(run + ["write-flash" if dashes else "write_flash", "0x0", args.image])
```

`flash.py` runs **esptool**, Espressif's flashing tool, with the same Python. `esptool` resets the chip into download mode (section 4.2), writes the image at flash offset `0x0`, where the ROM looks for a direct-boot program, and resets the board so the new program starts. Writing at offset 0 also replaces any ESP-IDF bootloader. `python tools/flash.py --erase` erases the whole flash first, which also removes anything kept elsewhere in it, such as a script saved with exercise 7.

monitor.py

A terminal for the Lua console. Every key goes to the board, including `Ctrl+C`, which stops a running Lua program; `Ctrl+J` quits. If the board resets or is unplugged, the monitor waits and asks for a fresh prompt when it comes back. It also takes piped input, typing each line once the board shows a prompt: `echo print(1 + 2) | python tools/monitor.py`

selftest.py

The self-test types Lua into the console and checks each answer, printing PASS or FAIL. Its exit status is 1 if anything failed. Leave GPIO4 unconnected (the test drives it), don't press the button, and close any monitor first. The 15 checks are:

1. C3U-Metal's Lua functions are there
2. `print(1 + 2)` prints 3
3. 32-bit integers and floats
4. a statement over several lines
5. errors print a message and a traceback
6. `Ctrl+C` stops a program that never ends
7. `delay(250)` takes 250 ms
8. `button()` is false while not pressed
9. GPIO4 as an output reads back 1, then 0
10. GPIO4 reads 1 with its pull-up, 0 with its pull-down
11. the USB pins are refused
12. `led(0, 0, 0)` is accepted
13. `led()` refuses 300
14. no `os` or `io` library (there are no files)
15. `mem()` works, and the stack has not overflowed

3.6 VS Code

Install the **C/C++** and **CMake Tools** extensions, which `.vscode/extensions.json` recommends (Espressif's **ESP-IDF** extension is optional; section 14.7 says what it is for), and start VS Code from the ESP-IDF terminal with `code .` **Terminal > Run Task** then lists Configure, Build, Flash, Monitor, Self-test and Read registers, which run the commands above. `.vscode/settings.json` points the C/C++ extension at `build/compile_commands.json` and tells CMake Tools to use the presets. Debugging is covered in section 14.7.

How it boots

Booting is everything that happens between power-on and the program doing its real work. For C3U-Metal it is short: the chip's ROM makes two checks and then jumps straight into our code.

4.1 After a reset, the ROM runs

When the board gets power, or is reset, the CPU starts running the **ROM**: code that Espressif built into the chip at address `0x40000000`. Nobody can change it, so it always works, whatever is in flash. At this moment it is the only code the chip knows about. It checks two things.

4.2 First check: download mode

Is GPIO9 low? On the C3U, GPIO9 is the button, and pressing the button connects the pin to ground. If the button is held while the chip starts, the ROM goes into **download mode** and waits for esptool to send a new program over USB.

A pin that the chip reads at reset to decide how to start is called a *strapping pin*, and the board's picture labels G9 that way. Because download mode happens before any of our code runs, it works whatever is on the board, even a program that crashes straight away. It is your way back from any mistake ([section 14.9](#)).

4.3 Second check: the direct-boot magic number

Does flash start with the direct-boot magic number? A *magic number* is a value chosen because it is very unlikely to appear by accident, so it can be used to recognise something. If the first 8 bytes of flash hold `0xAEDB041D` twice, the ROM does a **direct boot**:

1. It maps the first 4 MB of flash into the address space twice: at `0x42000000` for running code, and at `0x3C000000` for reading data.
2. It jumps to `0x42000008`, which is flash offset 8, just past the two magic numbers.

Our linker script puts the magic number and our first instruction exactly there:

`boot/c3u-metal.ld` the header and the entry point

lines 30–41

```

30  /* Flash offset 0: the ROM only runs our code if it finds this value twice. */
31  .header : AT(0)
32  {
33      LONG(0xAEDB041D)
34      LONG(0xAEDB041D)
35  } > irom
36
37  /* The ROM jumps to 0x42000008, so _start (section .text.entry) must be exactly here. */
38  .text.entry ORIGIN(irom) + 8 :
39  {
40      KEEP(*(.text.entry))
41  } > irom

```

- `.header : AT(0)` makes a section called `.header`, stored at offset 0 of the image.
- `LONG(0xAEDB041D)` puts four bytes holding that value into it, and the second line does it again.
- `> irom` places the section in the `irom` memory region, which starts at `0x42000000` (chapter 5).
- `.text.entry ORIGIN(irom) + 8` places the next section at exactly `0x42000008`.
- `KEEP(*(.text.entry))` collects the `.text.entry` section from every object file. `crt0.S` is the only file with one, and it holds `_start`. `KEEP` stops the linker throwing the section away, even though nothing in the program calls it.

Because `tools/flash.py` writes `c3u-metal.bin` at flash offset 0, the magic number ends up exactly where the ROM looks.

Try this

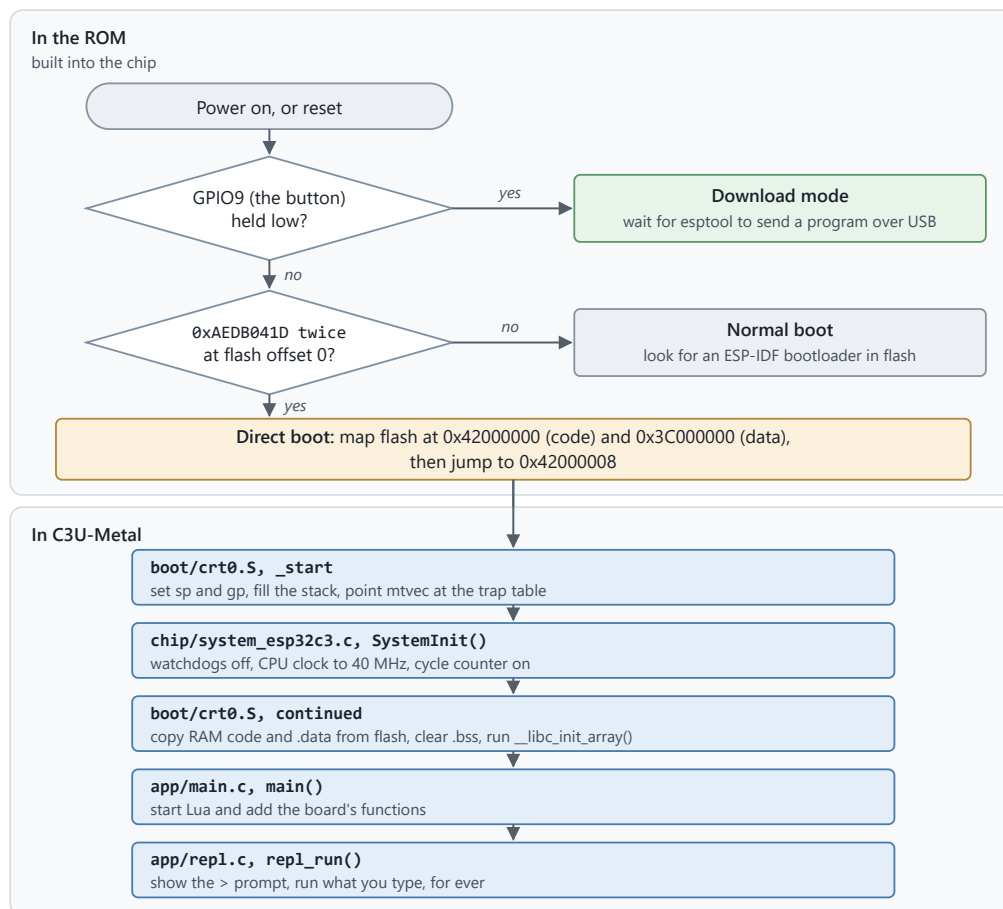
At the console, type `hex(peek(0x4200000))`. It reads the first word of flash through the code view and prints `0xaedb041d`, the magic number.

4.4 How ESP-IDF boots instead

If the magic number isn't there, the ROM boots the normal way. It loads a *second-stage bootloader* from flash. The bootloader sets up more of the chip, loads the application and starts FreeRTOS. Arduino programs for this chip start the same way.

C3U-Metal skips all of that. Nothing is lost by trying it: flashing an ordinary ESP-IDF or Arduino program writes a bootloader back to the start of flash, replacing C3U-Metal.

4.5 The whole boot, step by step



From reset to the Lua prompt. The top half is Espressif's ROM; everything in the bottom half is in the repository.

4.6 What the ROM leaves behind

When `_start` runs, almost nothing is ready for a program:

What	State at <code>_start</code>	Who fixes it
Stack pointer <code>sp</code> , global pointer <code>gp</code>	whatever the ROM left in them	<code>crt0.S</code> step 1
Trap vector <code>mtvec</code>	points into the ROM	<code>crt0.S</code> step 3
Three watchdogs	running : they reset the chip unless something stops or feeds them	<code>SystemInit()</code>
CPU clock	20 MHz, the 40 MHz crystal divided by 2	<code>SystemInit()</code>

What	State at <code>_start</code>	Who fixes it
RAM	leftovers: <code>.data</code> has no starting values and <code>.bss</code> isn't zero	<code>crt0.S</code> step 5

A **watchdog** is a timer that resets the chip if the program doesn't reset the timer, or "feed" it, often enough. It rescues a program that has got stuck. Chapter 7 shows how C3U-Metal deals with all three.

Why the order matters

- `SystemInit()` is written in C, so it needs a stack first.
- It runs before `.data` and `.bss` are ready, so it must not use global variables.
- The watchdogs are stopped early, before anything slow happens.

4.7 Watching the boot with a debugger

A debug session in VS Code ([section 14.7](#)) normally stops at `main()`, after the boot has finished. To watch the boot from the very first instruction, type these into VS Code's Debug Console:

Type in the VS Code Debug Console

```
-exec monitor reset halt
-exec thbreak _start
-exec continue
```

`monitor reset halt` asks OpenOCD to reset the chip and stop it before the ROM runs. `thbreak _start` sets a temporary hardware breakpoint on our first instruction, and `continue` lets the ROM run until it gets there. Then step one instruction at a time and watch `sp` and `gp` change.

The linker script and the memory map

The linker script `boot/c3u-metal.ld` decides where every part of the program lives: which bytes go in flash, which are copied into RAM, and where the stack and heap are. It is the first file the README suggests reading.

5.1 Sections: sorting a program into kinds of memory

A compiler doesn't decide where things go in memory. It sorts what it produces into named **sections**, and the linker places each section at an address. The main sections have standard names:

Section	What goes in it	Where it has to be
<code>.text</code>	machine code	where the CPU can run it: flash
<code>.rodata</code>	constants and strings	where it can be read: flash
<code>.data</code>	global and <code>static</code> variables with a starting value	in RAM, because they change; but their starting values must survive power-off, so they are stored in flash and copied at start-up
<code>.bss</code>	global and <code>static</code> variables that start at zero	in RAM; nothing is stored, the RAM is cleared at start-up
<code>.sdata</code> , <code>.sbss</code>	small variables from <code>.data</code> and <code>.bss</code>	the same, but near <code>gp</code> (5.7)
<code>.iram1</code>	C3U-Metal's own name for code that must run from RAM	in RAM; stored in flash and copied at start-up

Step 2 of the lessons in the git history had one variable of each kind, to check that its start-up code worked:

```
main.c at tag step-2-c-runtime one variable of each kind lines 24–32
24 /*
25  * One variable of each kind, so the runtime can be checked on the board:
26  * the LED only shows led_color if .data was copied, loop_count only counts up from
27  * 0 if .bss was cleared, and build_name is only readable if .rodata is mapped.
28  */
29 uint32_t led_color = 0x002000;          /* .data (0x00GGRRBB: dim red) */
30 uint32_t loop_count;                    /* .bss */
31 __attribute__((used))
32 static const char build_name[] = "C3U-Metal C runtime"; /* .rodata */
```

Local variables, declared inside a function, are not in any of these sections. They live on the stack while the function runs (chapter 6).

5.2 Where it runs and where it is stored

Look again at `.data`: its bytes are *stored* in one place (flash) but *used* in another (RAM). So a section can have two addresses. Linkers call them the **load address**, where the bytes are stored in the image, and the **run address**, where the program expects to find them while it runs. In a linker script, `AT(...)` sets the load address. The address written before the colon, or the memory region after `>`, sets the run address. For most sections the two are the same.

5.3 The MEMORY block

```
boot/c3u-metal.ld memory regions and constants lines 14–26
14 OUTPUT_ARCH("riscv")
15 ENTRY(_start)
16
17 MEMORY
18 {
19     irom (rx) : ORIGIN = 0x42000000, LENGTH = 4M /* flash, mapped for running code */
20     drom (r)  : ORIGIN = 0x3C000000, LENGTH = 4M /* the same flash, mapped for reading data */
21     ram (rw)  : ORIGIN = 0x3FC80000, LENGTH = 0x50000 /* SRAM for us; the ROM uses the area above */
```

```

22     iram (rwx) : ORIGIN = 0x40380000, LENGTH = 0x50000 /* the same SRAM, at the address it can run code from */
23 }
24
25 STACK_SIZE = 0x10000; /* 64 KB: Lua's parser recurses on the C stack */
26 IRAM_TO_DRAM = 0x40380000 - 0x3FC80000; /* 0x70000: same RAM byte, code address minus data address
    */

```

- `OUTPUT_ARCH` and `ENTRY(_start)` record the CPU type and the first instruction in the `.elf` file, for tools such as the debugger. The raw `.bin` image has no room for either; what the ROM relies on is the address `0x42000008`.
- `MEMORY` names four regions. The letters say what each allows: `r` read, `w` write, `x` execute (run code). `irom` and `drom` are the two views of the same flash. `ram` and `iram` are the two views of the same 320 KB of RAM (`0x50000` bytes).
- `STACK_SIZE` is 64 KB, because Lua's parser calls itself deeply and every call uses stack.
- `IRAM_TO_DRAM` is the distance between the two RAM views, `0x70000`.

5.4 Code and constants in flash

boot/c3u-metal.ld .text and .rodata

lines 43–62

```

43 /* All other code that runs straight from flash. */
44 .text :
45 {
46     *(.text .text.*)
47     . = ALIGN(4);
48 } > irom
49
50 /* How much flash the code uses: also the flash offset where .rodata starts. */
51 _flash_used = . - ORIGIN(irom);
52
53 /*
54  * .rodata: constants and strings. The CPU reads data through the DROM mapping,
55  * so the address is DROM + the flash offset, and AT() stores the bytes at that
56  * same flash offset in the image. The sections after it follow the same way.
57  */
58 .rodata ORIGIN(drom) + _flash_used : AT(_flash_used)
59 {
60     *(.rodata .rodata.* .srodata .srodata.*)
61     . = ALIGN(4);
62 } > drom

```

- `*(.text .text.*)` collects every `.text` section from every object file and library.
- `.` is the *location counter*, the address the linker has reached. `. = ALIGN(4)` moves it up to a multiple of 4.
- `_flash_used = . - ORIGIN(irom)` is how many bytes of flash the code takes, which is also the flash offset where the next section will be stored.

Now the trick. The byte at flash offset N can be read at `0x3C000000 + N`. So `.rodata` gets the run address `ORIGIN(drom) + _flash_used` and the load address `AT(_flash_used)`. Stored right after the code, it is read in place through the DROM view, and nothing needs copying.

After `.rodata` the script adds three small sections, `.preinit_array`, `.init_array` and `.fini_array`, which hold lists of functions. newlib's `__libc_init_array()` runs the functions listed in the first two before `main()`. C3U-Metal has none, but C++ constructors would go here. `PROVIDE_HIDDEN` defines the start and end symbols newlib looks for. `_drom_used` is the flash offset just past all the read-only data.

5.5 Code that runs from RAM

boot/c3u-metal.ld .iram.text

lines 89–102

```

89 /*
90  * .iram.text: code that must never wait on the flash cache (the LED's timed
91  * pulses in sk6812.S). It runs from RAM at the start of our area, is stored in flash after
92  * the read-only data, and crt0.S copies it into RAM before main().
93  */
94 .iram.text ORIGIN(iram) : AT(_drom_used)
95 {

```

```

96     _iram_text_start = .;
97     *(.iram1 .iram1.*)
98     . = ALIGN(4);
99     _iram_text_end = .;
100 } > iram
101 _iram_text_lma = ORIGIN(drom) + LOADADDR(.iram.text); /* crt0.S reads the code from here ... */
102 _iram_text_dram = _iram_text_start - IRAM_TO_DRAM; /* ... and writes it to the same RAM here */

```

`.iram.text` runs at `ORIGIN(iram)`, `0x40380000`, and is stored in flash at `_drom_used`. It collects sections named `.iram1`, which `drivers/sk6812.S` uses for `sk6812_send_grb`. Three addresses matter to the start-up code:

- `_iram_text_start` and `_iram_text_end`: the run addresses of the code.
- `_iram_text_lma`: where the stored copy can be read, through the DROM view.
- `_iram_text_dram`: the same RAM as `_iram_text_start`, but at its data address. The CPU runs code in RAM only from the instruction-bus address (`0x4038xxxx`), but writes to RAM go through the data-bus address (`0x3FC8xxxx`). `crt0.S` copies the code in through the data address, and the code later runs at the instruction address.

Why run the LED code from RAM?

Code in flash can pause for a moment while the chip's *flash cache* loads it. A pause in the middle of an LED pulse would change the colour. Code in RAM never waits (chapter 9).

5.6 Variables: `.data` and `.bss`

`boot/c3u-metal.ld` `.data` and `.bss`

lines 104–128

```

104 /*
105  * .data: variables with a starting value. They live in RAM just after the
106  * RAM code, and their starting values are stored in flash after it too;
107  * crt0.S copies them across.
108  */
109 .data ORIGIN(ram) + SIZEOF(.iram.text) : AT(LOADADDR(.iram.text) + SIZEOF(.iram.text))
110 {
111     _data_start = .;
112     *(.data .data.*)
113     __SDATA_BEGIN__ = .;
114     *(.sdata .sdata.*) /* small variables, reachable through gp */
115     . = ALIGN(4);
116     _data_end = .;
117 } > ram
118 _data_lma = ORIGIN(drom) + LOADADDR(.data); /* where crt0.S reads the starting values */
119
120 /* .bss: variables that start at zero. Nothing is stored in flash; crt0.S clears it. */
121 .bss (NOLOAD) :
122 {
123     _bss_start = .;
124     *(.sbss .sbss.*) /* small ones first, near gp */
125     *(.bss .bss.* COMMON)
126     . = ALIGN(4);
127     _bss_end = .;
128 } > ram

```

- `.data` runs in RAM straight after the RAM code, at `ORIGIN(ram) + SIZEOF(.iram.text)`. Its starting values are stored in flash straight after the RAM code's stored copy.
- `__SDATA_BEGIN__` marks where the small variables start. `_data_lma` is where `crt0.S` reads the starting values, through the DROM view.
- `.bss` is `NOLOAD`: it takes space in RAM but no bytes in the image. `COMMON` collects an old C style of uninitialised global variable.

5.7 The global pointer

```
boot/c3u-metal.ld __global_pointer$ lines 130–134
130 /*
131  * gp points into the small-variable area so GCC can reach variables within
132  * 2 KB of it in one instruction. This is the formula GNU ld uses by default.
133  */
134 __global_pointer$ = MIN(__SDATA_BEGIN__ + 0x800, MAX(_data_start + 0x800, _bss_end - 0x800));
```

Reading a global variable normally takes two instructions: one to load its 32-bit address, one to read from it. But if the variable is within 2 KB (0x800 bytes) either side of the address in `gp`, one instruction can reach it by an offset from `gp`. So `gp` is set once at start-up and never changes, and the linker puts the small variables near it. This formula is GNU ld's usual way of choosing `gp` so the small variables fit in that 4 KB window. In the example build below, `gp` is 0x3FC8086C.

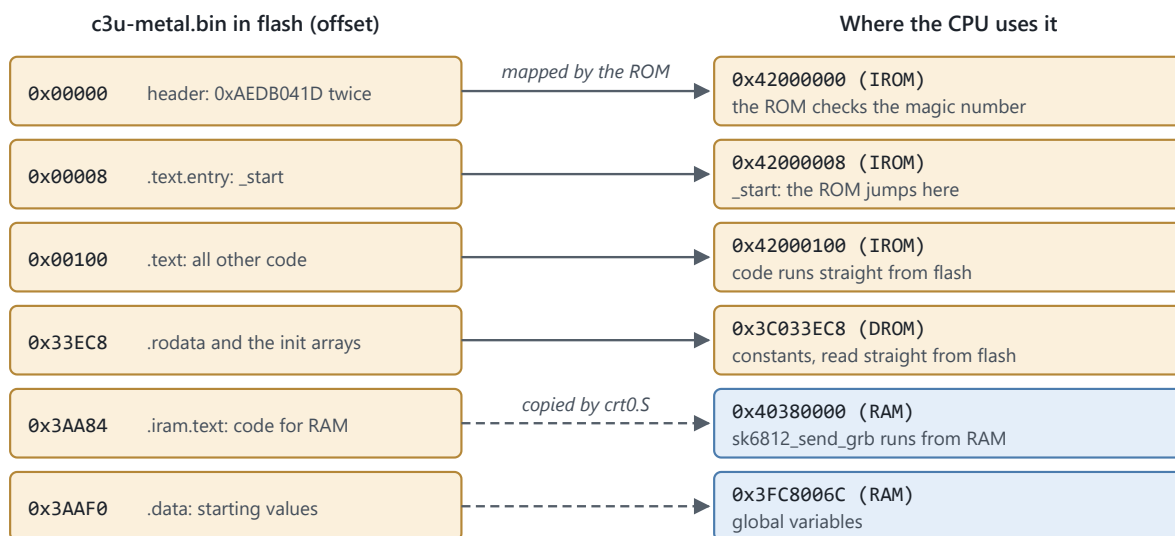
5.8 Stack and heap

```
boot/c3u-metal.ld stack, heap and discarded sections lines 136–146
136 /* Stack: the top STACK_SIZE bytes of our RAM, growing down. */
137 _stack_top = ORIGIN(ram) + LENGTH(ram);
138 _stack_bottom = _stack_top - STACK_SIZE;
139
140 /* Heap: everything between the variables and the stack, handed out by _sbrk() in syscalls.c. */
141 _heap_start = ALIGN(_bss_end, 8);
142 _heap_end = _stack_bottom;
143 ASSERT(_heap_end >= _heap_start + 0x20000, "c3u-metal.ld: less than 128 KB left for the heap")
144
145 /* Unwinding tables are only for C++ exceptions and debuggers that don't use DWARF. */
146 /DISCARD/ : { *(.eh_frame .eh_frame_hdr) }
```

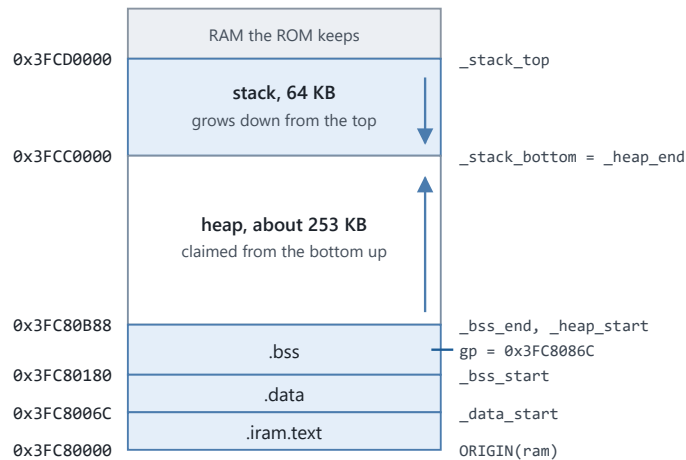
- The stack is the top 64 KB of our RAM. `_stack_top` is 0x3FCD0000, and the stack grows down from there.
- The **heap** is the memory `malloc()` hands out. It is everything between the end of `.bss` and the bottom of the stack. `ALIGN(_bss_end, 8)` starts it on a multiple of 8.
- `ASSERT` makes the link fail with that message if less than 128 KB is left for the heap.
- `/DISCARD/` throws away unwinding tables, which only C++ exceptions and some debuggers use.

5.9 The whole picture

These addresses come from one build, recorded in `docs/memory-map.md`. Yours will move a little as the code changes; `build/c3u-metal.map` has the exact values for your build.



One flash chip, several views. Solid arrows: the ROM's mapping makes the bytes usable where they are. Dashed arrows: `crt0.S` copies the bytes into RAM before `main()`.



C3U-Metal's 320 KB of RAM in the same build, bottom address at the bottom. Not to scale. The `.iram.text` bytes are also the code at `0x40380000`.

Nothing stops the stack growing down into the heap if a program uses more than 64 KB of stack. `boot/stack_check.c` can only tell afterwards (section 13.6).

5.10 Linker symbols in C

Names such as `_heap_start` are **linker symbols**: names for addresses, defined in the linker script. C code declares the ones it needs in `boot/linker_symbols.h`:

```
boot/linker_symbols.h linker symbols for C lines 11–12
11 extern char _heap_start[], _heap_end[];      /* the heap, handed out by _sbrk() in syscalls.c */
12 extern char _stack_bottom[], _stack_top[];   /* the stack grows down from _stack_top */
```

There is no variable stored at those addresses. Declaring the symbol as an array of `char` lets C use its name as an address without ever reading from it. Subtracting two of them gives a size in bytes: `main.c` prints the heap size as `( _heap_end - _heap_start ) / 1024` KB.

To see the real addresses after a build:

```
Run in a terminal, after a build
riscv32-esp-elf-objdump -h build/c3u-metal.elf      # every section: address, size, flash offset
riscv32-esp-elf-nm -n build/c3u-metal.elf | more   # every symbol, in address order
```

In `objdump -h`, *VMA* is the run address and *LMA* the load address. You can also search `build/c3u-metal.map` for any function or variable name, and at the console `mem()` shows how the heap and stack are being used.

crt0.S, the C runtime

`boot/crt0.S` holds the first instructions of our program. It is 50 lines of assembly that turn the chip's leftovers into a place where C code can run. The name is traditional: *C runtime, part zero*.

6.1 What C needs before `main()`

A C program takes several things for granted that a freshly reset chip doesn't have:

- A **stack**, for local variables and return addresses, with `sp` pointing at it.
- A **global pointer**, because the compiler uses `gp` to reach small global variables.
- **Global variables with their starting values** in `.data`, and zeros in `.bss`.
- **Anything else the C library expects to have run before `main()`**.

On a laptop, the operating system and the compiler's own start-up files do this. Here `crt0.S` does it, which is why `CMakeLists.txt` links with `-nostartfiles`.

6.2 Reading RISC-V assembly

Assembly language is a readable form of the CPU's instructions, one per line. You only need a handful to read `crt0.S`:

You see	It means
<code>_start:</code>	a <i>label</i> : a name for the address of the next instruction
<code>7: ... 7b, 8f</code>	numbered local labels. <code>7b</code> is the nearest <code>7:</code> <i>backwards</i> , <code>8f</code> the nearest <code>8:</code> <i>forwards</i>
<code>.section .text.entry, "ax"</code>	put what follows into this section; "ax" means it takes memory and can be executed
<code>.global _start</code>	let other files, and the linker script, see this label
<code>.option push / pop</code>	save the assembler's options, change them for a few lines, then restore them
<code>la sp, _stack_top</code>	<i>load address</i> : put the address of a symbol into a register
<code>li t1, STACK_FILL</code>	<i>load immediate</i> : put a number into a register
<code>sw t1, 0(t0)</code>	<i>store word</i> : write the 32 bits in <code>t1</code> to the address in <code>t0</code> , plus 0
<code>lw t3, 0(t0)</code>	<i>load word</i> : read 32 bits from the address in <code>t0</code> into <code>t3</code>
<code>addi t0, t0, 4</code>	<code>t0 = t0 + 4</code> . <code>add</code> and <code>sub</code> do the same with two registers
<code>ori t0, t0, 1</code>	<code>t0 = t0 1</code>
<code>j 8f</code>	jump
<code>bltu t0, sp, 7b</code>	<i>branch if less than, unsigned</i> : jump to <code>7b</code> if <code>t0 < sp</code>
<code>call SystemInit / ret</code>	call a function, setting <code>ra</code> so it can come back / go back to the address in <code>ra</code>
<code>csw mtvec, t0</code>	write a CSR (section 2.7)

A word is 32 bits, which is 4 bytes. That is why the copying loops below add 4 each time.

6.3 The plan

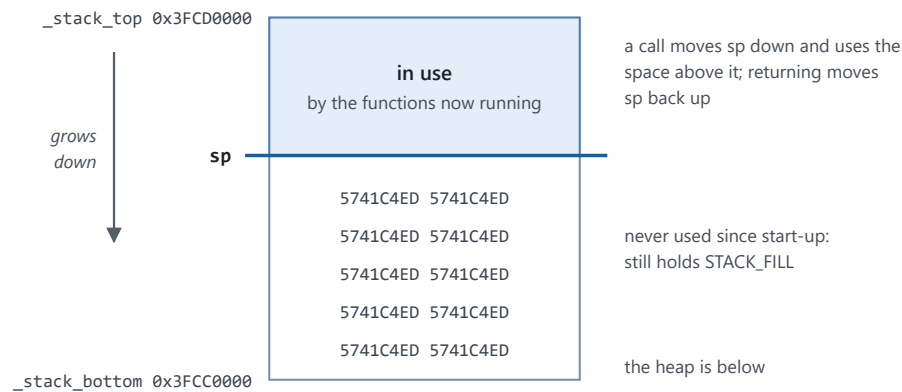
The comment at the top of the file lists the seven steps:

<code>boot/crt0.S</code> the seven steps		lines 8–14
8	* 1. <code>sp</code> and <code>gp</code>	so C functions can run
9	* 2. fill the stack	with <code>STACK_FILL</code> , so <code>stack_check.c</code> can measure its use
10	* 3. <code>mtvec</code>	traps go to the crash report (<code>chip/trap.S</code>)
11	* 4. <code>SystemInit()</code>	watchdogs, clock, cycle counter (no global variables yet)
12	* 5. RAM code, <code>.data</code> , <code>.bss</code>	copy code and starting values from flash; zero the rest
13	* 6. <code>__libc_init_array()</code>	<code>newlib</code> runs any start-up functions (constructors)
14	* 7. <code>main()</code>	

6.4 Step 1: the stack pointer and the global pointer

```
boot/crt0.S _start, step 1 lines 24–36
24      .section .text.entry, "ax"
25      .global _start
26      _start:
27      /*
28       * 1. Stack pointer and global pointer, both from the linker script.
29       *    "norelax" stops the linker rewriting these two instructions to be
30       *    relative to gp, which would use gp before it has a value.
31       */
32      .option push
33      .option norelax
34      la    sp, _stack_top
35      la    gp, __global_pointer$
36      .option pop
```

The **stack** is an area of memory that functions use as scratch space. When a function is called it moves `sp` down by as much space as it needs, keeps its local variables and saved registers there, and moves `sp` back up when it returns. The next call reuses the same space. Because every C function does this, the stack must exist before any C function runs. `la sp, _stack_top` points `sp` at the top of the stack area from the linker script.



The stack, top address at the top. Step 2 fills all of it with `STACK_FILL`, so the words that still hold that value have never been used.

The global pointer comes from the linker script too (section 5.7). The `.option norelax` lines matter. *Linker relaxation* is an optimisation in which the linker shortens instructions where it can, and one way it does that is by loading addresses relative to `gp`. For these two lines that would be a disaster, because `gp` has no value yet: the second line is the one setting it. `norelax` turns the optimisation off for just these lines, and `.option pop` turns it back on.

6.5 Step 2: painting the stack

```
boot/crt0.S _start, step 2 lines 38–44
38      /* 2. Fill the whole stack with STACK_FILL. Nothing has been pushed yet. */
39      la    t0, _stack_bottom
40      li    t1, STACK_FILL
41      j      8f
42 7:     sw    t1, 0(t0)
43      addi  t0, t0, 4
44 8:     bltu  t0, sp, 7b
```

```
boot/stack_check.h the fill value lines 13–13
13 #define STACK_FILL 0x5741C4ED /* any value unlikely to be pushed by chance */
```

This loop writes `STACK_FILL` into every word of the stack, from the bottom up to `sp`:

1. `t0` starts at the bottom of the stack and `t1` holds the fill value.
2. `j 8f` jumps straight to the test, so an empty range would do nothing.
3. At `7:` the value is stored at `t0`, and `t0` moves up one word.

4. At 8:, if `t0` is still below `sp`, the loop goes back to 7.

Later, any word that still holds `0x5741C4ED` has never been used. That lets `mem()` report the most stack ever used, and lets the console warn if the stack ran past its bottom (section 13.6). The file includes `stack_check.h` so the value is written in one place only.

6.6 Step 3: where traps go

boot/crt0.S _start, step 3

lines 46–49

```
46      /* 3. Traps (bad instruction, bad address) go to the crash report. +1 selects vectored mode. */
47      la    t0, trap_vector_table
48      ori   t0, t0, 1
49      csrw  mtvec, t0
```

A *trap* is the CPU abandoning what it was doing because something went wrong, such as an instruction it doesn't understand. The CPU then jumps to the address in the `mtvec` CSR. This step points `mtvec` at our trap table in `chip/trap.S`, so from now on any crash prints a report. The `+1`, done with `ori`, selects *vectored* mode. Chapter 14 explains both.

6.7 Step 4: SystemInit()

boot/crt0.S _start, step 4

lines 51–52

```
51      /* 4. Chip setup, in C. It runs before step 5, so it must not use global variables. */
52      call SystemInit
```

`SystemInit()` is a C function in `chip/system_esp32c3.c` that stops the watchdogs and speeds up the clock (chapter 7). It can be called now because there is a stack. It must not use global variables, because their starting values haven't been copied yet. Doing it this early stops the watchdogs before anything slow happens.

6.8 Step 5: copying and clearing memory

boot/crt0.S _start, step 5a: code for RAM

lines 54–70

```
54      /*
55      * 5a. RAM code (.iram.text): copy it from flash into RAM. It is written
56      *      through the RAM's data address and later runs at its code address;
57      *      both are the same memory.
58      */
59      la    t0, _iram_text_lma
60      la    t1, _iram_text_dram
61      la    t2, _iram_text_start
62      la    t3, _iram_text_end
63      sub   t3, t3, t2          /* t3 = bytes to copy */
64      add   t2, t1, t3          /* t2 = end address, as a data address */
65      j     6f
66 5:      lw    t4, 0(t0)
67          sw    t4, 0(t1)
68          addi t0, t0, 4
69          addi t1, t1, 4
70 6:      bltu  t1, t2, 5b
```

First the RAM code. The four `la` lines load the addresses from section 5.5:

- `t0`: where the code is stored, read through the DROM view.
- `t1`: where it goes, at the RAM's data address.
- `t2` and `t3`: the start and end of the code at its run address. `sub` turns them into a size, and `add` turns that into the end address on the data side.

The loop then copies one word at a time until `t1` reaches `t2`. Both are data addresses, so the comparison is fair.

boot/crt0.S _start, steps 5b and 5c: .data and .bss

lines 72–89

```
72      /* 5b. .data: copy the starting values from flash (read through DROM) into RAM */
73      la    t0, _data_lma
```

```

74     la    t1, _data_start
75     la    t2, _data_end
76     j     2f
77 1:    lw    t3, 0(t0)
78     sw    t3, 0(t1)
79     addi  t0, t0, 4
80     addi  t1, t1, 4
81 2:    bltu  t1, t2, 1b          /* the linker script keeps both ends multiples of 4 */
82
83     /* 5c. .bss: set every zero-start variable to zero */
84     la    t1, _bss_start
85     la    t2, _bss_end
86     j     4f
87 3:    sw    zero, 0(t1)
88     addi  t1, t1, 4
89 4:    bltu  t1, t2, 3b

```

The same kind of loop copies the starting values of `.data` from flash into RAM. The linker script keeps both ends on multiples of 4, so copying whole words never goes past the end. Finally, every word of `.bss` is set to zero by storing the `zero` register into it.

6.9 Steps 6 and 7: start-up functions and main()

boot/crt0.S `_start`, steps 6 and 7

lines 91–97

```

91     /* 6. Start-up functions listed in .init_array (none today, but C++ would need them) */
92     call  __libc_init_array
93
94     /* 7. Run the program. main() shouldn't return; if it does, stop here. */
95     call  main
96 halt:
97     j     halt

```

`__libc_init_array()` comes from `newlib`. It calls every function listed in `.preinit_array` and `.init_array`. There are none today, but C++ constructors would need it.

Then `call main`. `main()` should never return, because there is nothing to return to. If it does, the CPU reaches `halt:` and jumps to itself for ever, where a debugger can find it.

Try this

Use the debugger commands from section 4.7 to stop at `_start`, then step one instruction at a time. Watch `sp` and `gp` change in step 1, and watch the fill loop run.

SystemInit(): watchdogs, clock and cycle counter

`chip/system_esp32c3.c` holds everything specific to the ESP32-C3 that must happen before the program proper. It is the only file that knows about watchdogs and clock settings.

7.1 Why a separate SystemInit()?

Arm's CMSIS is a common way of organising start-up code for Arm microcontrollers. It keeps the start-up code generic: it prepares memory for C and calls a function named `SystemInit()`, in a file named `system_<device>.c`, for anything specific to the chip. C3U-Metal follows the same pattern on RISC-V, which is step 3 of the lessons (chapter 15). The benefit is a clean split: `crt0.S` would work on another RISC-V chip, and all the ESP32-C3 details are in one C file.

`chip/system_esp32c3.c` includes and a speed check

lines 16–22

```
16 #include "board.h"
17 #include "cpu.h"
18 #include "esp32c3-reg.h"
19
20 #if CPU_MHZ != 40
21 #error "SystemInit() only knows how to run the CPU at 40 MHz (the crystal, divided by 1)"
22 #endif
```

The `#if` is checked by the compiler, not at run time. If someone changes `CPU_MHZ` in `board.h`, the build stops with that message instead of producing a program whose timing is silently wrong. `drivers/sk6812.S` has the same check. Exercise 2 is about running at 160 MHz, which means updating those files properly.

7.2 Stopping the watchdogs

A watchdog resets the chip if the program doesn't feed it in time (section 4.6). The ROM leaves three running. An ESP-IDF program feeds them; C3U-Metal has nothing that would feed them regularly, so it switches off two and tells the chip to feed the third itself.

Watchdog registers are **write-protected**, so that a program running out of control can't switch them off by accident. Writing a *key*, a particular magic number, to the write-protect register unlocks them. Writing anything else locks them again.

`chip/esp32c3-reg.h` watchdog registers

lines 25–35

```
25 /* --- Watchdogs: RTC_CNTL block at 0x60008000, Timer Group 0 at 0x6001F000 --- */
26 #define RTC_CNTL_WDTCONFIG0      0x60008090 /* RTC watchdog config; bit 31 enable, bit 12 flash-boot mode */
27 #define RTC_CNTL_WDTWPROTECT     0x600080A8 /* write WDT_WKEY to unlock, anything else to lock */
28 #define RTC_CNTL_SWDT_CONF      0x600080AC /* super watchdog config */
29 #define RTC_CNTL_SWDT_WPROTECT   0x600080B0 /* write SWD_WKEY to unlock */
30 #define RTC_CNTL_SWDT_AUTO_FEED_EN 0x80000000 /* bit 31: the chip feeds the super watchdog itself */
31 #define TIMG0_WDTCONFIG0        0x6001F048 /* Timer Group 0 watchdog; bit 31 enable, bit 14 flash-boot mode */
32 #define TIMG0_WDTWPROTECT       0x6001F064 /* write WDT_WKEY to unlock */
33 #define TIMG0_WDT_CONF_UPDATE_EN 0x00400000 /* bit 22: apply the new watchdog config */
34 #define WDT_WKEY                0x50D83AA1 /* unlock key for the RTC and Timer Group watchdogs */
35 #define SWD_WKEY                0x8F1D312A /* unlock key for the super watchdog */
```

`chip/system_esp32c3.c` `SystemInit()`, steps 1 to 3

lines 24–39

```
24 void SystemInit(void)
25 {
26     /* 1. RTC watchdog: unlock, switch everything off (incl. flash-boot mode), lock */
27     REG(RTC_CNTL_WDTWPROTECT) = WDT_WKEY;
28     REG(RTC_CNTL_WDTCONFIG0) = 0;
29     REG(RTC_CNTL_WDTWPROTECT) = 0;
30
31     /* 2. Timer Group 0 watchdog: the same, plus the bit that applies the change */
32     REG(TIMG0_WDTWPROTECT) = WDT_WKEY;
33     REG(TIMG0_WDTCONFIG0) = TIMG0_WDT_CONF_UPDATE_EN;
34     REG(TIMG0_WDTWPROTECT) = 0;
35 }
```

```

36  /* 3. Super watchdog: let the chip feed it automatically, as ESP-IDF does */
37  REG(RTC_CNTL_SWD_WPROTECT) = SWD_WKEY;
38  REG(RTC_CNTL_SWD_CONF) |= RTC_CNTL_SWD_AUTO_FEED_EN;
39  REG(RTC_CNTL_SWD_WPROTECT) = 0;

```

1. **RTC watchdog.** Unlock with `WDT_WKEY`, write 0 to the configuration register, lock again by writing 0. Writing 0 clears every bit, including bit 31 (enable) and bit 12 (flash-boot mode), so the watchdog is off.
2. **Timer Group 0 watchdog.** The same, except that the value written has only bit 22 set, `TIMG_WDT_CONF_UPDATE_EN`. All the other bits are 0, so bit 31 (enable) and bit 14 (flash-boot mode) are off, and bit 22 tells this watchdog to apply its new configuration.
3. **Super watchdog.** It has its own key. Instead of switching it off, `|=` sets bit 31, which makes the chip feed it automatically, as ESP-IDF does.

Why `|=` in step 3 but `=` in steps 1 and 2?

`REG(x) |= bit` is short for `REG(x) = REG(x) | bit`: read the register, set one bit, write it back. The other bits keep their values. This is called **read-modify-write**. Steps 1 and 2 want every bit to be 0 apart from the update bit, so they write the whole value.

Try this

The file suggests commenting out step 1 or step 2, rebuilding and flashing. After a while the watchdog the ROM started resets the board, and the console drops out.

7.3 The CPU clock

The CPU clock is set by `SYSTEM_SYSCLK_CONF`, at `0x600C0058` in the `SYSTEM` block. The header describes its low 12 bits:

31 ... 12	11	10	9	8	7	6	5	4	3	2	1	0
other settings, kept	clock source 0 = crystal		divider – 1 (0 means divide by 1)									

chip/system_esp32c3.c `SystemInit()`, steps 4 and 5

lines 41–47

```

41  /* 4. CPU clock: stay on the 40 MHz crystal, but stop dividing it by 2.
42  *   Bits 11:10 = 0 selects the crystal; bits 9:0 = 0 means divide by 1. */
43  REG(SYSTEM_SYSCLK_CONF) &= ~0xFFFFu;
44
45  /* 5. Start the CPU cycle counter, used for LED pulses and delays */
46  cycle_counter_start();
47  }

```

`0xFFFFu` has bits 11 to 0 set. `~0xFFFFu` flips every bit, giving `0xFFFFF000`. `&=` with that value clears bits 11 to 0 and keeps the rest, which is read-modify-write again. Clearing bits 11:10 selects the 40 MHz crystal, and clearing bits 9:0 sets the divider to 1. Before this line the CPU ran at 20 MHz; after it, 40 MHz.

7.4 The cycle counter

Step 5 starts the cycle counter. Espressif added three CSRs to the ESP32-C3's CPU for counting events. They are not part of standard RISC-V. C3U-Metal uses them to count CPU cycles, and so to measure time:

chip/cpu.S `cycle_counter_start()` and `cycle_count()`

lines 12–24

```

12  /* void cycle_counter_start(void): make CSR_PCCR count CPU cycles (Espressif-specific CSRs) */
13  .global cycle_counter_start
14  cycle_counter_start:
15      li    t0, 1
16      csrw  CSR_PCER, t0          /* count CPU cycles */
17      csrw  CSR_PCMR, t0          /* start counting */
18      ret
19
20  /* uint32_t cycle_count(void): CPU cycles counted so far (wraps about every 107 s at 40 MHz) */
21  .global cycle_count

```

```

22 cycle_count:
23     csrr a0, CSR_PCCR
24     ret

```

- `cycle_counter_start` loads 1 into `t0`, writes it to `CSR_PCCR` to choose "count cycles", and to `CSR_PCMR` to start counting. `ret` returns.
- `cycle_count` reads `CSR_PCCR` into `a0`. By the calling convention a function's result goes in `a0`, so in C this is a function that returns the count.

These functions are in assembly because C can't produce `csrr` and `csrw`. C code calls them through the declarations in `chip/cpu.h`.

The count is 32 bits, so it can reach 4 294 967 295 before wrapping round to 0. At 40 million cycles a second that takes about 107 seconds.

7.5 Time since start-up

A clock that wraps every 107 seconds isn't much use for `millis()`, so `drivers/uptime.c` builds a 64-bit count from it:

drivers/uptime.c `uptime_cycles()` and `uptime_us()`

lines 17–31

```

17 uint64_t uptime_cycles(void)
18 {
19     static uint32_t last;
20     static uint64_t total;
21     uint32_t now = cycle_count();
22
23     total += (uint32_t)(now - last);    /* correct across one wrap of the counter */
24     last = now;
25     return total;
26 }
27
28 uint64_t uptime_us(void)
29 {
30     return uptime_cycles() / CPU_CYCLES_PER_US;
31 }

```

- `static` variables inside a function keep their value between calls. They live in `.bss`, so they start at 0.
- Each call reads the counter and adds the cycles since the previous call to a 64-bit `total`.
- `(uint32_t)(now - last)` works even if the counter wrapped in between. Unsigned subtraction in C wraps too: if `last` is `0xFFFFFFFF00` and `now` is `0x00000100`, then `now - last` is `0x200`, the true number of cycles.
- `uptime_us()` divides by `CPU_CYCLES_PER_US` (40) to get microseconds.

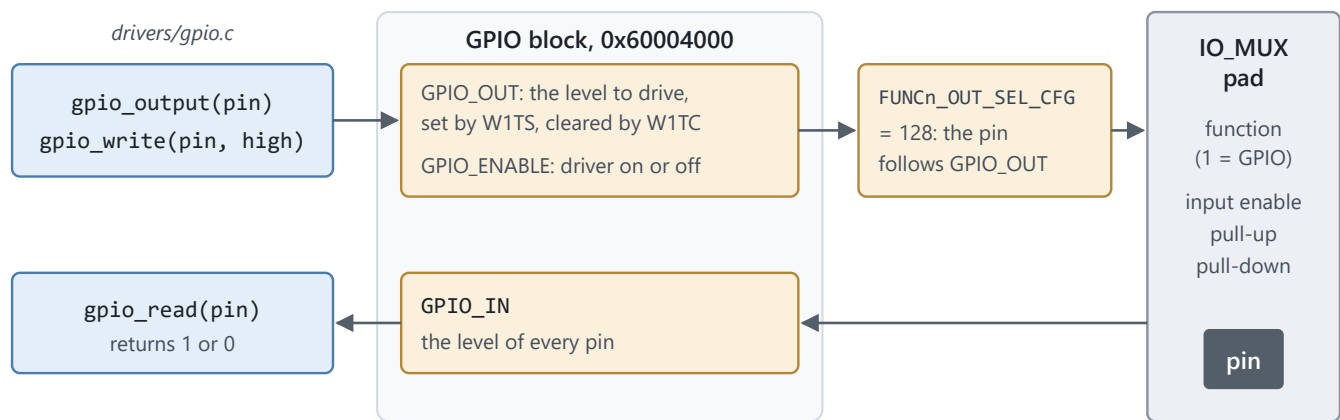
This only works if something calls `uptime_cycles()` at least once every 107 seconds, otherwise a whole wrap is missed. The console's idle loop, `delay()`, the Ctrl-C check and `_read()` all call it for that reason. Exercise 3 replaces this with the chip's SYSTIMER, a counter that keeps time by itself.

Registers and the GPIO driver

`drivers/gpio.c` makes the chip's pins into plain digital inputs and outputs. It is the clearest example in the project of a driver: a few functions that turn "make pin 4 an output" into the right bits in the right registers.

8.1 From a register bit to a pin

A **pad** is the piece of metal on the chip that connects to a pin. Several parts of the chip could drive the same pad, so two blocks of registers are involved, as the comment at the top of `drivers/gpio.c` explains. **IO_MUX** has one register per pin, which decides what its pad is connected to (plain GPIO, or a peripheral such as SPI) and switches the input buffer and pull resistors on or off. **GPIO** registers have one bit per pin: they set the output level, turn the output driver on or off, and read the level. For outputs, a per-pin `FUNCn_OUT_SEL_CFG` register picks which signal drives the pin.



Output (top): the program sets bits in the GPIO block, the output select passes `GPIO_OUT` to the pad, and the pad drives the pin.

Input (bottom): the pad's input buffer feeds the pin's level into `GPIO_IN`.

8.2 The registers

`chip/esp32c3-regs.h` IO_MUX and GPIO registers

lines 45–60

```

45 /* --- Pads: IO_MUX block at 0x60009000, one register per pin --- */
46 #define IO_MUX_GPIO(n)          (0x60009004 + 4 * (n)) /* the pad register for GPIOn */
47 #define IO_MUX_MCU_SEL_GPIO     (1 << 12) /* bits 14:12 function select; 1 = GPIO */
48 #define IO_MUX_FUN_IE           (1 << 9) /* input enable */
49 #define IO_MUX_FUN_PU           (1 << 8) /* pull-up */
50 #define IO_MUX_FUN_PD           (1 << 7) /* pull-down */
51 #define IO_MUX_CLEAR_MASK       0x7380 /* function select, input enable, pull-up and pull-down */
52
53 /* --- GPIO block at 0x60004000 --- */
54 #define GPIO_OUT_W1TS           0x60004008 /* write 1 to drive pins high */
55 #define GPIO_OUT_W1TC           0x6000400C /* write 1 to drive pins low */
56 #define GPIO_ENABLE_W1TS       0x60004024 /* write 1 to turn a pin's output driver on */
57 #define GPIO_ENABLE_W1TC       0x60004028 /* write 1 to turn a pin's output driver off */
58 #define GPIO_IN                 0x6000403C /* current level of every pin */
59 #define GPIO_FUNC_OUT_SEL_CFG(n) (0x60004554 + 4 * (n)) /* which signal drives GPIOn */
60 #define SIG_GPIO_OUT_IDX        128 /* signal number for "plain GPIO output" */

```

IO_MUX has one register for each pin. `IO_MUX_GPIO(n)` works out its address, and `GPIO_FUNC_OUT_SEL_CFG(n)` does the same for the output select. For example:

- `IO_MUX_GPIO(9) = 0x60009004 + 4 × 9 = 0x60009004 + 0x24 = 0x60009028`
- `GPIO_FUNC_OUT_SEL_CFG(2) = 0x60004554 + 4 × 2 = 0x6000455C`

Step 1 of the lessons wrote exactly those two addresses out by hand as `IO_MUX_GPIO9` and `GPIO_FUNC2_OUT_SEL_CFG`. The GPIO registers are different: they have one bit for each pin, so bit 4 of `GPIO_IN` is the level of GPIO4.

These are the bits of a pad register that the driver changes. The second row is `IO_MUX_CLEAR_MASK`, `0x7380`, in binary: a 1 for every bit the driver sets or clears.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
–	MCU_SEL (1 = GPIO)			–		IE	PU	PD	– (not used by this project)						
0	1	1	1	0	0	1	1	1	0	0	0	0	0	0	0

IE is `IO_MUX_FUN_IE` (input enable), PU is `IO_MUX_FUN_PU` (pull-up) and PD is `IO_MUX_FUN_PD` (pull-down).

8.3 Checking a pin number

drivers/gpio.c `gpio_check_pin()`
lines 22–35

```

22 #define GPIO_COUNT 22                      /* GPIO0 to GPIO21 */
23
24 const char *gpio_check_pin(int pin)
25 {
26     if (pin < 0 || pin >= GPIO_COUNT)
27         return "the ESP32-C3 has GPIO0 to GPIO21";
28     if (pin == 11)
29         return "GPIO11 powers the flash chip (VDD_SPI)";
30     if (pin >= 12 && pin <= 17)
31         return "GPIO12 to GPIO17 are connected to the flash chip";
32     if (pin == 18 || pin == 19)
33         return "GPIO18 and GPIO19 are the USB port, which the console uses";
34     return NULL;
35 }

```

Not every pin is free. GPIO11 and GPIO12 to GPIO17 belong to the flash chip that holds the program, and GPIO18 and GPIO19 are the USB port the console runs over. Changing those would crash the program or cut off the console. `gpio_check_pin()` returns `NULL` for a usable pin, or a message explaining why not. The other functions don't check, so the Lua functions call this first with any number a user typed (section 13.4). `gpio.output()` also refuses GPIO9, the button, for a reason of its own; reading GPIO9 is still allowed.

8.4 Configuring the pad

drivers/gpio.c `pad_config()`
lines 37–43

```

37 /* Connect the pad to GPIO and set its input-enable and pull-resistor bits. */
38 static void pad_config(int pin, uint32_t bits)
39 {
40     uint32_t pad = IO_MUX_GPIO(pin);
41
42     REG(pad) = (REG(pad) & ~IO_MUX_CLEAR_MASK) | IO_MUX_MCU_SEL_GPIO | bits;
43 }

```

This is read-modify-write on one line. `REG(pad) & ~IO_MUX_CLEAR_MASK` reads the register and clears the six bits that matter. `| IO_MUX_MCU_SEL_GPIO` sets the function field to 1, plain GPIO. `| bits` adds whichever of input enable, pull-up and pull-down the caller asked for. The result is written back, and every other bit keeps its old value. `static` means only this file can call the function.

8.5 Outputs

drivers/gpio.c `gpio_output()`
lines 45–51

```

45 void gpio_output(int pin)
46 {
47     pad_config(pin, IO_MUX_FUN_IE);          /* input stays on, so gpio_read() sees what we drive */
48     REG(GPIO_FUNC_OUT_SEL_CFG(pin)) = SIG_GPIO_OUT_IDX;
49     REG(GPIO_OUT_W1TC) = 1u << pin;          /* low first ... */
50     REG(GPIO_ENABLE_W1TS) = 1u << pin;        /* ... then turn the driver on */
51 }

```

1. Connect the pad to GPIO, with its input left on, so that `gpio_read()` can see the level being driven.
2. Choose signal 128, `SIG_GPIO_OUT_IDX`: the pin follows the `GPIO_OUT` register.
3. Set the pin's bit in `GPIO_OUT` to 0.
4. Only then switch the output driver on.

The order of the last two steps matters. If the driver came on first, the pin would briefly drive whatever level `GPIO_OUT` already held, and that could be high.

8.6 Inputs and pull resistors

`drivers/gpio.h` names the three choices `GPIO_PULL_NONE`, `GPIO_PULL_UP` and `GPIO_PULL_DOWN`. An input with nothing connected to it *floats*: it picks up electrical noise and can read 0 or 1 at random. A **pull-up** is a weak resistor inside the chip (about 45 k Ω , says `gpio.h`) that connects the pin to the supply voltage, so it reads 1 until something stronger pulls it to ground. A **pull-down** does the opposite. The button uses a pull-up: the pin reads 1, and pressing the button connects it to ground so it reads 0.

```
drivers/gpio.c gpio_input() lines 53–63
53 void gpio_input(int pin, enum gpio_pull pull)
54 {
55     uint32_t bits = IO_MUX_FUN_IE;
56
57     if (pull == GPIO_PULL_UP)
58         bits |= IO_MUX_FUN_PU;
59     else if (pull == GPIO_PULL_DOWN)
60         bits |= IO_MUX_FUN_PD;
61     REG(GPIO_ENABLE_W1TC) = 1u << pin;      /* output driver off */
62     pad_config(pin, bits);
63 }
```

8.7 Writing and reading

```
drivers/gpio.c gpio_write() and gpio_read() lines 65–78
65 void gpio_write(int pin, bool high)
66 {
67     /* W1TS/W1TC: "write 1 to set" and "write 1 to clear" change only the pins
68        whose bits are 1, so no read-modify-write is needed. */
69     if (high)
70         REG(GPIO_OUT_W1TS) = 1u << pin;
71     else
72         REG(GPIO_OUT_W1TC) = 1u << pin;
73 }
74
75 bool gpio_read(int pin)
76 {
77     return (REG(GPIO_IN) >> pin) & 1u;
78 }
```

`GPIO_OUT_W1TS` and `GPIO_OUT_W1TC` change `GPIO_OUT` only in the bits you write as 1. **W1TS** means *write 1 to set*: writing `1u << 4` sets bit 4 and leaves the other pins alone. **W1TC**, *write 1 to clear*, clears them. No read is needed, so a change is one write: faster, and safe even if another part of the program changes a different pin at the same moment, which matters once interrupts exist (exercise 4). `gpio_read()` shifts `GPIO_IN` right so the pin's bit lands in bit 0, then keeps only that bit.

Try this

Connect an LED and a 330 Ω resistor in series from GPIO4 to GND, with the LED's longer leg towards GPIO4, and type `gpio.output(4) gpio.write(4, 1)`. `gpio.read()` returns 1 or 0, and in Lua 0 counts as true, so write `if gpio.read(4) == 1` then, not `if gpio.read(4)` then.

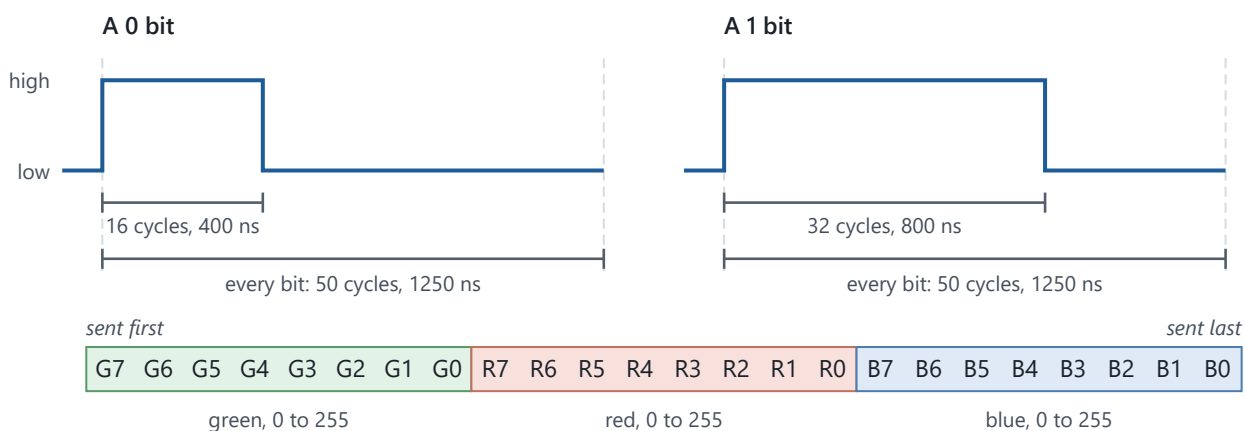
The SK6812 RGB LED

The board's RGB LED is controlled through a single wire, with pulses that must be timed to a few hundred nanoseconds. `drivers/sk6812.S` does it in assembly, from RAM, using the cycle counter as a stopwatch.

9.1 One wire, 24 bits

The SK6812 is an *addressable* RGB LED: a red, a green and a blue LED and a tiny controller chip in one package. It has a single data input, connected to GPIO2. The controller measures how long the line stays high at the start of each bit: a short pulse is a 0 and a long pulse is a 1. Every bit takes the same total time.

A colour is 24 bits: 8 for green, then 8 for red, then 8 for blue, each sent most significant bit first. Holding the line low for a while tells the LED that the next bits start a new colour. At start-up, `sk6812_init()` in `drivers/sk6812.c` makes the pin an output, holds it low for 1 ms, and sends colour 0 to turn the LED off.



Top: the two kinds of bit, in CPU cycles at 40 MHz (from `sk6812.S`). Bottom: the order of the 24 bits in one colour. All 24 take about 30 microseconds.

The Lua function `led(r, g, b)` packs the three values into one number in that order:

`app/lua_hw.c` `l_led()`

lines 48–57

```
48 /* led(r, g, b) */
49 static int l_led(lua_State *L)
50 {
51     uint32_t r = channel(L, 1);
52     uint32_t g = channel(L, 2);
53     uint32_t b = channel(L, 3);
54
55     sk6812_send_grb((g << 16) | (r << 8) | b); /* the LED wants green, red, blue */
56     return 0;
57 }
```

`g << 16` moves green into bits 23 to 16, `r << 8` moves red into bits 15 to 8, and blue stays in bits 7 to 0. So `led(0, 0, 40)` sends `0x000028`, and `led(40, 0, 0)` sends `0x002800`.

9.2 Timing in CPU cycles

drivers/sk6812.S timing constants

lines 18–26

```
18 #if CPU_MHZ != 40
19 #error "sk6812.S: HIGH_LATENCY and BIT_LATENCY were measured at 40 MHz and must be measured again"
20 #endif
21
22 /* SK6812 bit timing in CPU cycles (one cycle = 25 ns at 40 MHz) */
23 #define T0H      (CPU_MHZ * 400 / 1000) /* "0" bit high: 400 ns = 16 cycles */
24 #define T1H      (CPU_MHZ * 800 / 1000) /* "1" bit high: 800 ns = 32 cycles */
25 #define TBIT      (CPU_MHZ * 1250 / 1000) /* every bit: 1250 ns = 50 cycles */
26 #define TRESET    (CPU_MHZ * 100) /* low between colours: 100 us; the LED latches after 80 us */
```

The timings are worked out by the preprocessor from `CPU_MHZ`. At 40 MHz, $400 * 400 / 1000$ is 16 cycles. The `#error` stops the build if the CPU speed changes, because the corrections in section 9.5 were measured at 40 MHz.

9.3 Why assembly, and why RAM

The comments above `sk6812_send_grb` set out three rules:

- **Assembly**, because each pulse is timed to a few CPU cycles. A C compiler is free to change how many instructions a loop takes, and a different compiler version or optimisation setting could change the timing.
- **Nothing to interrupt it**. An interrupt in the middle of a pulse would stretch it. No interrupt source is connected to the CPU in this program, so nothing can. Exercise 4 adds interrupts, and turns them off inside this function.
- **From RAM**. `.section .iram1` puts the function in the section the linker script runs from RAM (section 5.5). Code in flash can stall while the flash cache loads it.

9.4 Walking through sk6812_send_grb

drivers/sk6812.S sk6812_send_grb: setting up, then sending each bit

lines 56–99

```
56     .section .iram1, "ax"
57     .global sk6812_send_grb
58 sk6812_send_grb:
59     /*
60      * The LED only takes in a colour once its data line has been low for
61      * 80 us. Calls can come faster than that (a Lua loop manages one every
62      * 55 us), so first wait until the previous colour is TRESET cycles old.
63      */
64     la    t0, sk6812_last_end
65     lw    t1, 0(t0)
66     li    t2, TRESET
67 9:      csrr t3, CSR_PCCR
68         sub t3, t3, t1
69         bltu t3, t2, 9b
70
71     li    t0, GPIO_OUT_W1TS
72     li    t1, GPIO_OUT_W1TC
73     li    t2, 1 << LED_PIN
74     li    t3, TBIT - BIT_LATENCY
75     li    a2, 24 /* bits left */
76     sllli a0, a0, 8 /* line bit 23 up with the sign bit */
77     csrr t4, CSR_PCCR
78
79 next_bit:
80     li    t5, T0H - HIGH_LATENCY /* high time for a 0 ... */
81     bgez a0, 1f
82     li    t5, T1H - HIGH_LATENCY /* ... or for a 1 */
83 1:      csrr a1, CSR_PCCR /* a1 = cycle count as this bit starts */
84         sw    t2, 0(t0) /* pin high */
85         sub   a4, a1, t4 /* a4 = length of the previous bit */
86         mv    t4, a1
87 2:      csrr a1, CSR_PCCR
88         sub   a1, a1, t4
89         bltu a1, t5, 2b /* wait out the high time */
90         sw    t2, 0(t1) /* pin low */
91         csrr a1, CSR_PCCR
92         sub   a3, a1, t4 /* a3 = high time of this bit */
93 3:      csrr a1, CSR_PCCR
```

```

94     sub  a1, a1, t4
95     bltu a1, t3, 3b           /* wait out the rest of the bit */
96
97     slli a0, a0, 1           /* next bit */
98     addi a2, a2, -1
99     bnez a2, next_bit

```

- Before anything else, a short loop waits until the previous colour is `TRESET` cycles, 100 μ s, old. The LED only takes in a colour once its data line has been low for 80 μ s, and Lua can call `led()` every 55 μ s, so without this wait a quick loop of colours would show only the first of them. `sk6812_last_end`, stored just before `ret`, is when the previous colour finished; the unsigned subtraction stays right across a wrap of the cycle counter, as in `uptime.c`.
- `t0` and `t1` hold the addresses of `GPIO_OUT_W1TS` and `GPIO_OUT_W1TC`, and `t2` holds the bit for the LED's pin. Loading them now keeps the timed loop short.
- `t3` is how long a whole bit should wait, and `a2` counts the bits left to send.
- `slli a0, a0, 8` shifts the colour left by 8 bits, so bit 23, the first to send, lands in bit 31. Bit 31 is the *sign bit*: when it is 1 the number counts as negative. Now "is the next bit a 1?" is the same question as "is `a0` negative?", which one instruction can answer.
- `t4` gets the cycle count, a starting point for the first measurement.

Then the loop from `next_bit` sends the bits:

1. `t5` is set to the high time for a 0. `bgez` (branch if greater than or equal to zero) skips the next line if the top bit is 0; otherwise `t5` becomes the high time for a 1.
2. Read the cycle count into `a1`, then drive the pin high with one `sw` to the `W1TS` register.
3. `a4 = a1 - t4` measures the previous bit's length, and `t4 = a1` records when this bit started.
4. Loop 2: reads the counter, subtracts the start, and repeats while fewer than `t5` cycles have passed.
5. Drive the pin low, then measure the high time into `a3`.
6. Loop 3: waits until the whole bit time `t3` has passed since the start.
7. Shift `a0` left so the next bit is in bit 31, count down `a2`, and go round again until all 24 bits are sent.

Once the colour has been sent, the lines after the loop store the last measurements in two global variables in `.bss`, where a debugger can read them, and the cycle count in `sk6812_last_end`, for the wait at the top of the next call.

9.5 Calibrating the timing

The wait loops compare the cycle count with a target, but reading the counter, comparing and writing the pin take instructions too, and every instruction takes at least a cycle. So each wait lasts a little longer than asked. The file records how that was measured and corrected:

drivers/sk6812.5 latency corrections

lines 28–38

```

28 /*
29  * The wait loops, and the instructions around them, take a few cycles longer
30  * than the number they wait for. Measured on the board with a debugger with no
31  * correction: high time 22 cycles instead of 16, bit length 66 instead of 50.
32  * The waits are shortened by those amounts. With these values the first,
33  * all-assembly version (lesson step 1) measured 18 cycles high and 48 per bit;
34  * this driver measures 17 cycles high (425 ns) and 51 per bit (1275 ns). Both
35  * include an instruction or two of the measuring code.
36  */
37 #define HIGH_LATENCY    6
38 #define BIT_LATENCY    16

```

With no correction, a 0 bit's high time came out at 22 cycles instead of 16 and a bit took 66 instead of 50. So the targets are reduced by 6 and by 16. With those corrections, the first all-assembly version (lesson step 1) measured 18 cycles high and 48 per bit, and this driver measures 17 cycles high (425 ns) and 51 per bit (1275 ns). Both include an instruction or two of the measuring code. This way of fixing timing is called latency calibration: measure the delay your own code adds, then subtract it.

Run in a terminal, with the program running on the board

```
python tools/read_registers.py sk6812_last_high sk6812_last_bit
```

This reads the two variables over JTAG ([section 14.6](#)). Expect about 17 and 51. The measurement instructions are part of the loop that was calibrated, which is why they stay in even though the LED doesn't need them.

Try this

Set `HIGH_LATENCY` to 0, flash, and measure again. Exercise 8 does the same job with the chip's RMT peripheral, which makes the pulses in hardware.

The USB console driver

The console needs no extra chip and no USB software stack. The ESP32-C3 has USB built in, and `drivers/usb_serial.c` talks to it through three registers.

10.1 USB built into the chip

The ESP32-C3's USB Serial/JTAG peripheral makes the board appear on your computer as two devices at once:

- a serial port, such as `COM11` on Windows or `/dev/ttyACM0` on Linux, used by the console;
- a JTAG debug adapter, used by OpenOCD and the debugger (chapter 14).

The hardware handles USB itself, and it keeps working whatever program is running. The board's USB pins, GPIO18 and GPIO19, are connected to it, which is why `gpio_check_pin()` refuses them.

10.2 Three registers

chip/esp32c3-regs.h USB Serial/JTAG registers

lines 62–67

```
62 /* --- USB Serial/JTAG block at 0x60043000 (usb_serial_jtag_reg.h) --- */
63 #define USB_SERIAL_EP1          0x60043000 /* read: next received byte; write: add a byte to send */
64 #define USB_SERIAL_EP1_CONF     0x60043004
65 #define USB_SERIAL_WR_DONE      (1 << 0) /* write 1: send the buffered bytes now */
66 #define USB_SERIAL_TX_FREE      (1 << 1) /* reads 1: room in the send buffer (SERIAL_IN_EP_DATA_FREE) */
67 #define USB_SERIAL_RX_AVAIL     (1 << 2) /* reads 1: a received byte is waiting (SERIAL_OUT_EP_DATA_AVAIL) */
```

EP1 stands for endpoint 1. In USB, an endpoint is one channel of data between the computer and a device.

10.3 Sending bytes

The driver keeps three `static` variables between calls: `pending`, the number of bytes in the 64-byte send buffer; `last_packet_full`, whether the hardware has just sent a full packet by itself; and `nobody_listening`, whether the computer seems to have stopped listening.

drivers/usb_serial.c `usb_serial_putc()`

lines 54–65

```
54 void usb_serial_putc(char c)
55 {
56     if (!wait_for_room())
57         return;
58     REG(USB_SERIAL_EP1) = (uint8_t)c;
59     pending++;
60     last_packet_full = false;
61     if (pending == PACKET_SIZE) { /* a full buffer is sent automatically */
62         pending = 0;
63         last_packet_full = true;
64     }
65 }
```

Each write to `EP1` adds one byte to the buffer. When the buffer holds 64 bytes, the hardware sends it by itself, so the driver resets its count and notes that a full packet went out.

drivers/usb_serial.c `usb_serial_write()`

lines 84–92

```
84 void usb_serial_write(const char *buf, size_t len)
85 {
86     for (size_t i = 0; i < len; i++) {
87         if (buf[i] == '\n' && (i == 0 || buf[i - 1] != '\r'))
88             usb_serial_putc('\r'); /* terminals expect CR LF; keep one that is already there */
89         usb_serial_putc(buf[i]);
90     }
91     usb_serial_flush();
92 }
```

Terminals expect a line to end with two characters: carriage return (`\r`, back to the start of the line) and line feed (`\n`, down a line). C programs only write `\n`, so the driver adds the `\r`, unless the text already has one there. After the whole buffer of text, it flushes.

10.4 Waiting for room, but not for ever

drivers/usb_serial.c `wait_for_room()`

lines 35–52

```
35 static bool wait_for_room(void)
36 {
37     if (REG(USB_SERIAL_EP1_CONF) & USB_SERIAL_TX_FREE) {
38         nobody_listening = false;
39         return true;
40     }
41     if (nobody_listening)
42         return false;
43
44     uint32_t start = cycle_count();
45     while (!(REG(USB_SERIAL_EP1_CONF) & USB_SERIAL_TX_FREE)) {
46         if (cycle_count() - start >= TX_TIMEOUT_CYCLES) {
47             nobody_listening = true;
48             return false;
49         }
50     }
51     return true;
52 }
```

The computer only collects packets while a terminal has the port open. With no terminal, the send buffer fills up and never empties. A driver that simply waited for room would wait for ever, and the board would freeze.

So `wait_for_room()` waits at most 50 ms, measured with the cycle counter. After a timeout it sets `nobody_listening` and stops waiting altogether, dropping output, until it sees room in the buffer again. Otherwise every single character would cost 50 ms while the terminal is closed. This is why the start-up banner is lost if no terminal is open: press Enter for a fresh prompt.

10.5 Ending a transfer

drivers/usb_serial.c `usb_serial_flush()`

lines 67–82

```
67 void usb_serial_flush(void)
68 {
69     if (pending > 0) {
70         REG(USB_SERIAL_EP1_CONF) = USB_SERIAL_WR_DONE;
71         pending = 0;
72     } else if (last_packet_full) {
73         /*
74          * A packet of exactly 64 bytes tells the PC "more is coming", so the
75          * terminal waits. An empty packet (WR_DONE with nothing buffered)
76          * ends the transfer and makes it show the data.
77          */
78         if (wait_for_room())
79             REG(USB_SERIAL_EP1_CONF) = USB_SERIAL_WR_DONE;
80         last_packet_full = false;
81     }
82 }
```

If bytes are waiting, writing `WR_DONE` sends them as one packet. The `else if` part handles a detail that took some debugging. USB marks the end of a transfer with a packet *shorter* than the largest size, 64 bytes here. A full 64-byte packet tells the computer that more is coming, so it may hold back the data and wait. After a full packet, the driver therefore sends an *empty* packet, `WR_DONE` with nothing in the buffer, which means "that's all".

Try this

`print(string.rep("x", 62)) delay(3000)` sends exactly 64 bytes, counting the `\r\n`, and then waits. Remove the `else if (last_packet_full)` part of `usb_serial_flush()` and try again: the line may not appear until the prompt follows it.

10.6 Receiving bytes

`drivers/usb_serial.c` `usb_serial_getc()`

lines 94–99

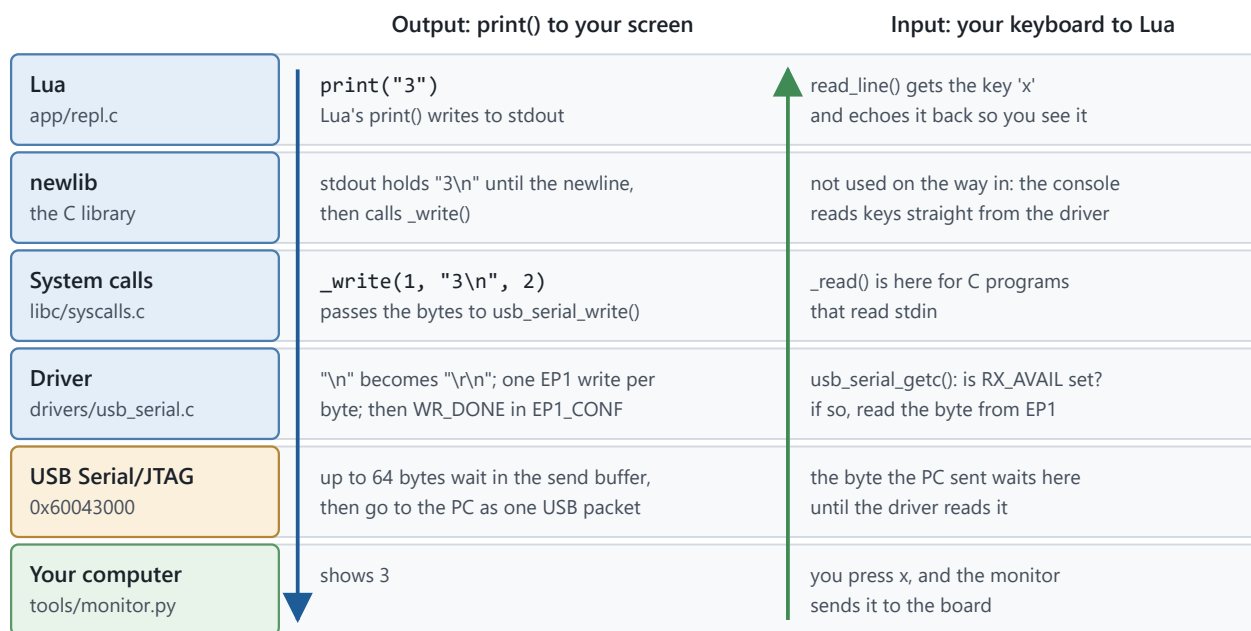
```
94 int usb_serial_getc(void)
95 {
96     if (REG(USB_SERIAL_EP1_CONF) & USB_SERIAL_RX_AVAIL)
97         return REG(USB_SERIAL_EP1) & 0xFF;
98     return -1;
99 }
```

Receiving is simpler. If `RX_AVAIL` is set, read one byte from `EP1`; otherwise return `-1`. Nothing interrupts the program when a key arrives. Instead the console calls `usb_serial_getc()` again and again while it waits. Checking over and over like this is called **polling**. The alternative is an **interrupt**, where the hardware makes the CPU run a function as soon as something happens; exercise 5 changes the driver to work that way.

While Lua code runs, the console checks for `Ctrl+C` every 1000 Lua instructions and keeps any other keys for the next line (section 12.7).

10.7 The whole path

This diagram follows one line of output from Lua down to your screen, and one key back up. The layers between Lua and the driver are the subject of the next chapter.



Output goes down the left, input comes up the right. The computer asks the board for data about every millisecond.

10.8 DTR and RTS

A serial port has two control lines, DTR and RTS. On ESP32 boards with a separate USB-to-serial chip, those lines are wired to the ESP32's reset and boot pins, and `esptool` wiggles them to reset the chip into download mode. The USB Serial/JTAG peripheral copies this behaviour. That is how `flash.py` can reset a running board, but it also means that a terminal program which changes DTR or RTS can reset the board by accident. `tools/board.py` opens the port with both lines off, which leaves the running program alone.

newlib and syscalls.c

C3U-Metal doesn't write its own `printf` or `malloc`. It uses newlib, the C library that comes with the compiler, and gives newlib the handful of functions it expects an operating system to provide.

11.1 What a C library needs

A **C library** provides the standard functions every C programmer expects: `printf`, `malloc`, `strlen`, `sin` and so on. **newlib** is a C library designed for small systems, and a copy built for the ESP32-C3 comes with the toolchain.

Most of newlib is ordinary code that works anywhere: formatting a number as text, comparing strings, doing maths. But a few jobs can't be done without help. To move bytes to a screen or get more memory, newlib calls functions that on a laptop would ask the operating system. These are called **system calls**, and newlib names them with a leading underscore: `_write`, `_read`, `_sbrk` and so on. On C3U-Metal they are all in `libc/syscalls.c`.

The link line in `CMakeLists.txt` (section 3.3) adds newlib as `c`, with `m` for its maths functions and `gcc` for compiler helpers. Because the link uses `-nostdlib`, nothing is added that the build doesn't ask for.

11.2 How printf reaches the screen

- 1 `printf()`, or Lua's `print()`, formats the text and hands it to `stdout`, one of newlib's `FILE` streams.
- 2 `stdout` keeps the text in a buffer. Because the console is a character device (below), newlib *line-buffers* it: the buffer is written out at the end of each line, when it is full, or when someone calls `fflush(stdout)`.
- 3 To write the buffer, newlib calls `_write(1, buffer, length)`. File descriptor 1 means `stdout`.
- 4 `_write()` passes the bytes to `usb_serial_write()`, which sends them over USB (chapter 10).

libc/syscalls.c `_write()`

lines 45–54

```
45 /* stdout (1) and stderr (2) go to the USB console. */
46 int _write(int fd, const char *buf, int len)
47 {
48     if (fd != 1 && fd != 2) {
49         errno = EBADF;
50         return -1;
51     }
52     usb_serial_write(buf, (size_t)len);
53     return len;
54 }
```

A *file descriptor* is a small number that stands for an open file: 0 is `stdin`, 1 is `stdout` and 2 is `stderr`. Both output streams go to the USB console. Anything else fails: `errno`, the C library's error code, is set to `EBADF` ("bad file descriptor") and the function returns -1.

libc/syscalls.c `_fstat()`

lines 73–82

```
73 /* The three console streams are character devices, which makes newlib line-buffer stdout. */
74 int _fstat(int fd, struct stat *st)
75 {
76     if (fd >= 0 && fd <= 2) {
77         st->st_mode = S_IFCHR;
78         return 0;
79     }
80     errno = EBADF;
81     return -1;
82 }
```

`_fstat()`, and `_isatty()` just below it, tell newlib that descriptors 0 to 2 are *character devices*, terminals rather than files. That is what makes newlib line-buffer `stdout`, so text appears line by line instead of in large chunks. The console's `push_line()` calls `fflush(stdout)` before each prompt so that anything Lua printed without a newline still appears first.

11.3 Reading keys: `_read()`

`_read()` serves `stdin`. It waits in a loop until a byte arrives, then returns just that one byte:

libc/syscalls.c `_read()`, waiting for a key

lines 67–68

```
67 while ((c = usb_serial_getc()) < 0)
68     uptime_cycles();           /* nothing typed yet; keep the uptime count right */
```

While it waits it calls `uptime_cycles()`, as the console's idle loop does, so a program that waits a long time for a key doesn't miss a wrap of the cycle counter (section 7.5). A read is allowed to return fewer bytes than were asked for, and returning one at a time keeps the console responsive. The console itself doesn't use this: `repl.c` reads keys directly from the driver so it can handle Backspace and `Ctrl+C`. `_read()` is there for C code that uses `getchar()` or `scanf()`.

11.4 Where malloc gets memory: `_sbrk()`

`malloc()` hands out blocks of memory from the heap and takes them back with `free()`. It keeps track of the blocks itself. When it needs more memory than it has, it calls `_sbrk()` to claim more of the heap. The boundary between claimed and unclaimed heap is traditionally called the **break**, hence the name.

libc/syscalls.c `_sbrk()`

lines 31–43

```
31 /* malloc() asks for more memory with this: move the end of the used heap by 'increment' bytes. */
32 void *_sbrk(ptrdiff_t increment)
33 {
34     static char *brk = _heap_start;
35     char *old = brk;
36
37     if (increment > _heap_end - brk || increment < _heap_start - brk) {
38         errno = ENOMEM;
39         return (void *)-1;
40     }
41     brk += increment;
42     return old;
43 }
```

- `brk` is the current break. It is `static`, so it keeps its value between calls, and it starts at `_heap_start`, the bottom of the heap from the linker script. Because it has a starting value it lives in `.data`, and `crt0.S` copies that value in before `main()`.
- The `if` refuses any move past `_heap_end`, which would go into the stack, or below `_heap_start`. It sets `errno` to `ENOMEM` ("not enough memory") and returns `(void *)-1`, the traditional failure value. `malloc()` then returns `NULL`.
- Otherwise the break moves by `increment`, and the function returns the *old* break: the start of the newly claimed memory.

Other C files call `_sbrk()` too, so it is declared in `libc/syscalls.h`. `mem()` uses `_sbrk(0)` to find out how much of the heap has never been claimed (section 13.6).

11.5 Only one task: `__getreent()`

libc/syscalls.c `__getreent()`

lines 25–29

```
25 /* newlib as built for ESP-IDF finds its per-task state through this. There is only one "task". */
26 struct _reent *__getreent(void)
27 {
28     return _impure_ptr;
29 }
```

newlib keeps some of its state, such as `errno`, in a `struct _reent`. With an operating system there could be one for each task. The build of newlib made for ESP-IDF asks `__getreent()` for the current task's structure. C3U-Metal has only one "task", so it returns newlib's global one, `_impure_ptr`.

11.6 Things that don't exist here

newlib's code refers to system calls for files and processes. Even if the program never opens a file, the linker needs every function that the parts of newlib it uses refer to. So `syscalls.c` provides small *stubs*, functions that do nothing but report a sensible error:

libc/syscalls.c `_open()`, one of the stubs

lines 92–98

```
92 /* There are no files. */
93 int _open(const char *path, int flags, int mode)
94 {
95     (void)path; (void)flags; (void)mode;
96     errno = ENOENT;
97     return -1;
98 }
```

Function	What it does here	errno
<code>_open</code> , <code>_unlink</code>	there are no files	ENOENT: no such file
<code>_close</code>	nothing is ever open	EBADF
<code>_lseek</code>	the console can't seek	ESPIPE
<code>_fcntl</code>	not supported; newlib's <code>fopen</code> refers to it	ENOSYS
<code>_link</code>	no files to link	EMLINK
<code>_getpid</code>	returns 1, the only "process"	
<code>_kill</code>	nothing to signal; <code>abort()</code> ends up here	EINVAL
<code>_exit</code>	loops for ever: there is nothing to return to	

The `(void)path;` lines tell the compiler that ignoring a parameter is deliberate, which silences a warning.

11.7 Time

libc/syscalls.c `_gettimeofday()`

lines 156–167

```
156 /* time() and gettimeofday(): time since start-up (there is no real-time clock). */
157 int _gettimeofday(struct timeval *tv, void *tz)
158 {
159     (void)tz;
160     uint64_t us = uptime_us();
161
162     if (tv) {
163         tv->tv_sec = (time_t)(us / 1000000u);
164         tv->tv_usec = (suseconds_t)(us % 1000000u);
165     }
166     return 0;
167 }
```

The board has no real-time clock, so the time is the time since start-up, from `uptime_us()` ([section 7.5](#)).

`_gettimeofday()` splits the microseconds into seconds and the microseconds left over. It serves C's `time()` and `gettimeofday()`. `_times()` serves `clock()`: newlib adds its fields together and treats the result as `CLOCKS_PER_SEC` ticks a second, so all the time goes in `tms_utime`. Lua's `os.clock()` would use `clock()`, but the `os` library isn't opened on this board.

Lua and the REPL

Lua turns the board into something you can talk to. This chapter shows how C3U-Metal starts Lua, how C and Lua pass values to each other, and how `app/repl.c` reads, runs and prints each line you type.

12.1 Lua on the board

Lua is a small programming language designed to be built into other programs. It is written in plain C and needs nothing from an operating system beyond what `newlib` and `syscalls.c` already provide, so Lua 5.5.1 runs on C3U-Metal unmodified, from `third_party/lua`. It is *interpreted*: you type code and it runs at once, with no compiling or flashing. A **REPL** (read, evaluate, print, loop) is the kind of prompt it runs in, and C3U-Metal's follows `lua.c`, the standard Lua interpreter, closely.

Lua is compiled with `LUA_32BITS` (section 3.3), which makes its integers and floating-point numbers 32 bits instead of 64. That suits a 32-bit chip with little RAM, and you can see it at the prompt: `math.maxinteger` is 2147483647, integers wrap round after that, and `millis()` wraps after about 24 days. The setting is `PUBLIC` in CMake, so our own C files are compiled with it too; otherwise our code and Lua would disagree about what a Lua number is.

12.2 Starting Lua: main.c

lines 53–69

```

app/main.c main()
53 int main(void)
54 {
55     lua_State *L = lua_start();
56
57     if (L == NULL) {
58         printf("C3U-Metal: not enough memory to start Lua\n");
59         for (;;) {
60             }
61     }
62     lua_hw_open(L);                /* led, button, delay, millis, gpio */
63     lua_sys_open(L);              /* help, mem, peek, hex, crash */
64
65     printf("\n%s\nC3U-Metal: %d KB heap. Type help() to see what this board adds to Lua.\n"
66           "Press Enter for a prompt.\n",
67           LUA_COPYRIGHT, (int)((_heap_end - _heap_start) / 1024));
68     repl_run(L);
69 }

```

`lua_start()`, just above `main()` in the same file, calls `luaL_newstate()` to create an interpreter. It gets its memory from `malloc()`, so it returns `NULL` if the heap is too small. Then it opens six standard libraries with `luaL_requiref()`: the basic functions, coroutines, tables, strings, maths and UTF-8. `io`, `os`, `package` and `debug` are left out, so `print(os, io)` prints `nil nil`. It also removes `dofile()` and `loadfile()` from the basic library: there are no files for them to load, and called without a name they would read Lua from the console instead, leaving no prompt. `main()` then adds the board's functions (chapter 13), prints a banner with the heap size, and hands over to `repl_run()`, which never returns.

12.3 The Lua C API in five ideas

The **Lua C API** is the set of C functions that let C code and Lua work together. Chapter 4 of Lua's reference manual, `third_party/lua/doc/manual.html`, describes all of it; five ideas are enough here.

- 1. The state.** A `lua_State *`, always called `L`, is the whole interpreter. Every API function takes it first.
- 2. The stack.** Values pass between C and Lua on a stack of Lua values, separate from the C stack. Positions count from 1 at the bottom, or from -1 at the top. When Lua calls `led(0, 0, 40)`, the C function finds `0` at index 1 (or -3), `0` at index 2 (or -2) and `40` at index 3 (or -1, the top).

3. **Push, read and check.** `lua_pushinteger()`, `lua_pushboolean()` and `lua_pushstring()` add a value on top. `lua_tostring()` reads one without removing it. `luaL_checkinteger()` reads an argument and raises an error if it isn't an integer. `lua_settop(L, 0)` empties the stack.

4. Many API names are macros, defined in `lua.h` in terms of other functions:

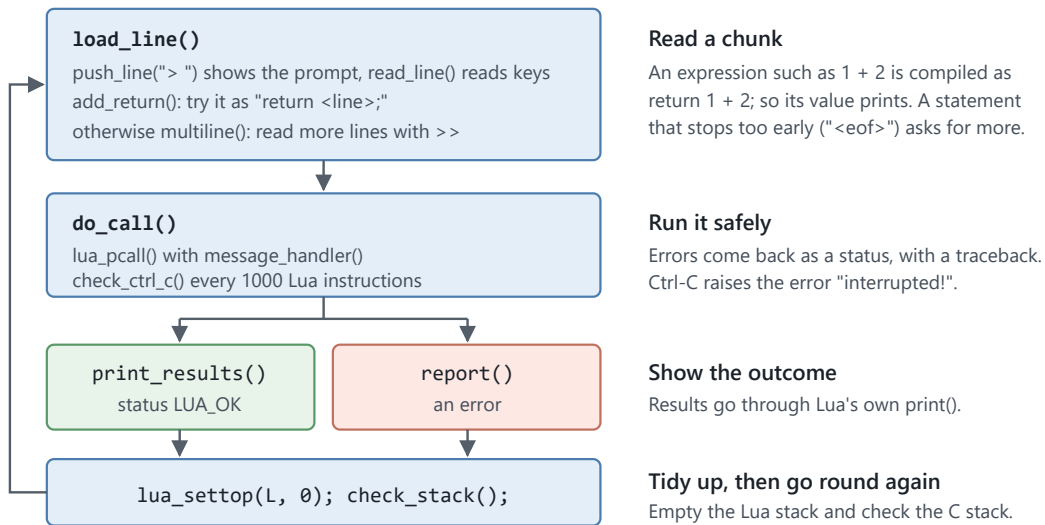
third_party/lua/src/lua.h	macros used by C3U-Metal	lines 396–402
396	<code>#define lua_pop(L,n) lua_settop(L, -(n)-1)</code>	
397		
398	<code>#define lua_newtable(L) lua_createtable(L, 0, 0)</code>	
399		
400	<code>#define lua_register(L,n,f) (lua_pushcfunction(L, (f)), lua_setglobal(L, (n)))</code>	
401		
402	<code>#define lua_pushcfunction(L,f) lua_pushcclosure(L, (f), 0)</code>	

So `lua_register(L, "led", l_led)` pushes the C function and stores it in the global variable `led`.

5. **Errors jump.** `luaL_error()` and `luaL_argerror()` raise a Lua error, and the C function doesn't carry on after the call. To run Lua code without an error ending everything, call it with `lua_pcall()`, a *protected call*: it returns a status, `LUA_OK` or an error code, and leaves the error message on the stack.

12.4 The REPL at a glance

Settings at the top of `repl.c` name the prompts, `>` and `>>`, limit a line to 512 bytes as `lua.c` does, and set `HOOK EVERY` to 1000 Lua instructions between Ctrl-C checks. Each trip round the loop has four phases:



One trip round `repl_run()`, which loops for ever.

12.5 Reading a line

`read_line()` collects keys into a buffer until Enter. It starts by handling two things: when no key has arrived, it calls `uptime_cycles()` to keep the clock right (section 7.5) and tries again; and it skips *escape sequences*, the several bytes starting with ESC (27) that arrow and function keys send. The rest of the loop handles ordinary keys:

app/repl.c	<code>read_line()</code> : Enter, Backspace, Ctrl-C and ordinary keys	lines 92–118
92	<code>if (c == '\n' && prev == '\r')</code>	
93	<code>continue;</code>	<code>/* LF right after CR: same Enter */</code>
94	<code>if (c == '\r' c == '\n') {</code>	
95	<code>usb_serial_write("\n", 1);</code>	
96	<code>buf[len] = '\0';</code>	
97	<code>return len;</code>	
98	<code>}</code>	
99	<code>if (c == '\b' c == 0x7F) {</code>	<code>/* Backspace or Delete */</code>
100	<code>if (len > 0) {</code>	

```

101         len--;
102         usb_serial_write("\b \b", 3);
103     }
104     continue;
105 }
106 if (c == CTRL_C) {           /* throw the line away */
107     usb_serial_write("^C\n", 3);
108     buf[0] = '\0';
109     return 0;
110 }
111 if (c < ' ')
112     continue;               /* other control keys */
113 if (len + 1 < size) {
114     buf[len++] = (char)c;
115     usb_serial_putc((char)c);
116     usb_serial_flush();
117 }
118 }

```

- **Enter:** terminals send `\r`, `\n` or both. `prev` is the key before, so `\r` followed by `\n` counts as one Enter. The line gets its closing 0 byte and its length is returned.
- **Backspace or Delete:** remove the last character and send `"\b \b"`: move back, overwrite with a space, move back again.
- **Ctrl+C:** print `^C` and return an empty line. Other control keys are ignored.
- **Anything else:** if there is room, keeping one byte for the closing 0, store it and echo it so you see what you typed.

12.6 Expression or statement?

You might type an expression, such as `1 + 2`, or a statement, such as `x = 1` or the first line of a loop. `load_line()` tries both, the way `lua.c` does:

app/repl.c `add_return()` lines 148–160

```

148 /* Try to compile the line on the stack as "return <line>". On success the chunk replaces the line. */
149 static int add_return(lua_State *L)
150 {
151     const char *line = lua_tostring(L, -1);
152     const char *retline = lua_pushfstring(L, "return %s;", line);
153     int status = luaL_loadbufferx(L, retline, strlen(retline), "=stdin", "t");
154
155     if (status == LUA_OK)
156         lua_remove(L, -2);           /* remove the "return" version of the text */
157     else
158         lua_pop(L, 2);             /* remove the error and the "return" version */
159     return status;
160 }

```

`add_return()` builds the text `return 1 + 2`; and compiles it with `luaL_loadbufferx()`. The chunk name `"=stdin"` appears in error messages, and mode `"t"` accepts only text, not precompiled Lua. If it compiles, the compiled function replaces the text on the stack; if not, the error and the `return` version are removed.

app/repl.c `multiline()` and `load_line()` lines 170–201

```

170 /* Compile the line on the stack as a statement, reading more lines while it is unfinished. */
171 static int multiline(lua_State *L)
172 {
173     size_t len;
174     const char *line = lua_tolstring(L, 1, &len);
175
176     check_local(line);
177     for (;;) {
178         int status = luaL_loadbufferx(L, line, len, "=stdin", "t");
179         if (!incomplete(L, status))
180             return status;           /* compiled, or a real error */
181         lua_pop(L, 1);               /* remove the "<eof>" error message */
182         push_line(L, PROMPT2);
183         lua_pushliteral(L, "\n");    /* join: first line, newline, next line */
184         lua_insert(L, -2);
185         lua_concat(L, 3);
186         line = lua_tolstring(L, 1, &len);

```

```

187     }
188 }
189
190 /* Read a complete chunk. Leaves the compiled function, or an error message, on the stack. */
191 static int load_line(lua_State *L)
192 {
193     int status;
194
195     lua_settop(L, 0);
196     push_line(L, PROMPT);
197     if ((status = add_return(L)) != LUA_OK)
198         status = multiline(L);
199     lua_remove(L, 1);
200     return status;
201 }

```

Otherwise `multiline()` compiles the line as a statement. When a statement stops too early, as for `i = 1, 3` does, Lua's syntax error ends with `<eof>` (end of file), which `incomplete()` spots. Then `multiline()` removes the error, shows the `>>` prompt, and joins the next line on. After `push_line()` the stack holds the first line and the new line. `lua_pushliteral()` adds `"\n"` on top, `lua_insert(L, -2)` moves it between them, and `lua_concat(L, 3)` joins all three into one string. The loop tries again until the statement compiles or has a real error. Finally `load_line()` removes the source text at index 1, leaving just the compiled function or an error message.

`check_local()` prints a warning if a line starts with `local`, because each line is compiled as its own chunk, so a local variable is gone by the next line.

12.7 Running a chunk, and Ctrl-C

app/repl.c `do_call()` lines 241–253

```

241 /* Call the function on the stack with the message handler and Ctrl-C checks in place. */
242 static int do_call(lua_State *L, int nargs, int nresults)
243 {
244     int base = lua_gettop(L) - nargs;
245
246     lua_pushcfunction(L, message_handler);
247     lua_insert(L, base);
248     lua_sethook(L, check_ctrl_c, LUA_MASKCOUNT, HOOK_EVERY);
249     int status = lua_pcall(L, nargs, nresults, base);
250     lua_sethook(L, NULL, 0, 0);
251     lua_remove(L, base);
252     return status;
253 }

```

- `base` is the stack position of the function to call. `message_handler` is pushed and moved underneath it.
- `lua_sethook()` asks Lua to call `check_ctrl_c()` every 1000 instructions.
- `lua_pcall()` runs the function. `LUA_MULTRET` keeps all its results, and `base` names the message handler.
- Afterwards the hook is removed and the handler is taken off the stack, leaving the results or the error message.

When an error happens, Lua calls `message_handler()` before unwinding the stack, so it can still see where the error came from. Like the one in `lua.c`, it uses `luaL_traceback()` to add a *traceback*: the list of function calls that led to the error.

app/repl.c `repl_check_interrupt()` and `check_ctrl_c()` lines 206–225

```

206 void repl_check_interrupt(lua_State *L)
207 {
208     int c;
209
210     uptime_cycles();
211     while ((c = usb_serial_getc()) >= 0) {
212         if (c == CTRL_C) {
213             lua_sethook(L, NULL, 0, 0);
214             luaL_error(L, "interrupted!");
215         }
216         keep_key(c);
217     }
218 }

```

```

219
220 /* Called by Lua every HOOK_EVERY instructions while a chunk runs. */
221 static void check_ctrl_c(lua_State *L, lua_Debug *ar)
222 {
223     (void)ar;
224     repl_check_interrupt(L);
225 }

```

The hook reads every key that has arrived. `Ctrl+C` removes the hook and raises the error `interrupted!`, which `lua_pcall()` catches. Other keys go to `keep_key()`, which stores up to 64 of them for the next line in a **ring buffer**: an array used as a queue, where the position of the next free place wraps round to the start with `%`, so nothing ever needs shuffling. `next_key()` takes keys from there before asking the driver. Slow C functions such as `delay()` call `repl_check_interrupt()` themselves, so `Ctrl+C` works inside them too.

12.8 Printing results and the main loop

app/repl.c `repl_run()` lines 289–302

```

289 void repl_run(lua_State *L)
290 {
291     for (;;) {
292         int status = load_line(L);
293         if (status == LUA_OK)
294             status = do_call(L, 0, LUA_MULTRET);
295         if (status == LUA_OK)
296             print_results(L);
297         else
298             report(L);
299         lua_settop(L, 0);
300         check_stack();
301     }
302 }

```

The loop follows the diagram: load, call, print or report, tidy up. If the chunk left values on the stack, `print_results()` fetches Lua's global `print`, slides it underneath them and calls it, so results look exactly as in standard Lua. `report()` prints an error message to `stderr`. `check_stack()` is covered in [section 14.8](#).

Try this

Type `for i = 1, 3 do`, then press `Ctrl+C`. The line is thrown away, but the unfinished statement isn't, so you stay at the `>>` prompt. Exercise 6 fixes that.

The board's Lua functions

`app/lua_hw.c` gives Lua the LED, the button, time and the pins. `app/lua_sys.c` adds functions about the system itself. Each is a short C function, and together they are a good model for writing your own.

13.1 A C function that Lua can call

`app/lua_hw.c` `l_button()`

lines 59–64

```
59 /* button() -> boolean */
60 static int l_button(lua_State *L)
61 {
62     lua_pushboolean(L, !gpio_read(BTN_PIN));    /* pulled up, so low means pressed */
63     return 1;
64 }
```

Every function Lua can call has this shape. As the header of `lua_hw.c` puts it, each is "an ordinary C function with Lua's calling convention: arguments are read from the Lua stack, results are pushed onto it, and the return value is how many results there are." The button's pin has a pull-up, so `gpio_read(BTN_PIN)` is 1 when the button is *not* pressed. `!` turns that round, `lua_pushboolean()` pushes true or false, and `return 1` says there is one result.

13.2 Checking arguments: `led()`

`app/lua_hw.c` `channel()`

lines 39–46

```
39 /* Read argument 'arg' as a colour channel, or raise a Lua error if it isn't 0-255. */
40 static uint32_t channel(lua_State *L, int arg)
41 {
42     lua_Integer value = luaL_checkinteger(L, arg);
43
44     luaL_argcheck(L, value >= 0 && value <= 255, arg, "must be 0-255");
45     return (uint32_t)value;
46 }
```

`l_led()` (section 9.1) calls `channel()` for each argument. `luaL_checkinteger()` raises an error if the argument is missing or not an integer, so `led("red", 0, 0)` fails. `luaL_argcheck()` raises an error naming the argument if its condition is false, which is why `led(300, 0, 0)` prints a message containing `must be 0-255`. These errors don't crash anything: they end the C function, `lua_pcall()` in the console catches them, and the console prints them.

13.3 `delay()` and `millis()`

`app/lua_hw.c` `l_delay()` and `l_millis()`

lines 66–83

```
66 /* delay(ms) */
67 static int l_delay(lua_State *L)
68 {
69     lua_Integer ms = luaL_checkinteger(L, 1);
70
71     luaL_argcheck(L, ms >= 0, 1, "must not be negative");
72     uint64_t end = uptime_cycles() + (uint64_t)ms * CPU_CYCLES_PER_MS;
73     while (uptime_cycles() < end)
74         repl_check_interrupt(L);    /* Ctrl-C raises an error and ends the wait */
75     return 0;
76 }
77
78 /* millis() -> integer */
79 static int l_millis(lua_State *L)
80 {
81     lua_pushinteger(L, (lua_Integer)((uptime_us() / 1000u) & (uint64_t)LUA_MAXINTEGER));
82     return 1;
83 }
```

`delay(ms)` refuses negative numbers, works out the cycle count to wait for, and loops until then. This is a *busy wait*: the CPU runs the loop the whole time. Each time round it calls `repl_check_interrupt()`, which keeps the uptime count right and lets `Ctrl+C` end the wait. `millis()` turns microseconds into milliseconds; & `LUA_MAXINTEGER` keeps the result positive, so it wraps to 0 after about 24 days. Try `t = millis() delay(1000) print(millis() - t)`: each of those milliseconds is 40 000 CPU cycles.

13.4 The gpio table

app/lua_hw.c check_pin(), check_free_pin() and l_gpio_output()

lines 88–119

```

88 /* Read argument 'arg' as a pin number, or raise a Lua error saying why that pin can't be used. */
89 static int check_pin(lua_State *L, int arg)
90 {
91     int pin = (int)lual_checkinteger(L, arg);
92     const char *problem = gpio_check_pin(pin);
93
94     if (problem != NULL)
95         luaL_argerror(L, arg, problem);
96     return pin;
97 }
98
99 /*
100  * Read argument 'arg' as a pin that Lua may reconfigure: any usable pin except
101  * the button's and the LED's. The button connects its pin to ground, so driving
102  * that pin high while it is pressed would short the output. Changing the LED's
103  * pin would stop led() working until the board restarts. Reading either is fine.
104  */
105 static int check_free_pin(lua_State *L, int arg)
106 {
107     int pin = check_pin(L, arg);
108
109     luaL_argcheck(L, pin != BTN_PIN, arg, "that pin is the button; driving it could short it to ground");
110     luaL_argcheck(L, pin != LED_PIN, arg, "that pin drives the RGB LED; use led()");
111     return pin;
112 }
113
114 /* gpio.output(pin) */
115 static int l_gpio_output(lua_State *L)
116 {
117     gpio_output(check_free_pin(L, 1));
118     return 0;
119 }

```

Every `gpio` function starts with `check_pin()`. If `gpio_check_pin()` says the pin can't be used, `luaL_argerror()` raises an error that includes its explanation, which is how `gpio.output(18)` says the pin belongs to the USB port.

`gpio.output()` and `gpio.input()` go through `check_free_pin()` as well, which refuses two more pins. GPIO9 is the button: it connects the pin to ground, so driving the pin high while the button is pressed would short the output. GPIO2 drives the LED: reconfiguring it would stop `led()` working until the board restarts. Reading either pin is still allowed.

`gpio.input()` uses `lual_checkoption()` to turn "none", "up" or "down" into a position in a list, with "none" if the argument is missing. `gpio.write()` accepts `true` and `false` as well as 1 and 0.

app/lua_hw.c gpio_functions and lua_hw_open()

lines 156–177

```

156 static const luaL_Reg gpio_functions[] = {
157     { "output", l_gpio_output },
158     { "input", l_gpio_input },
159     { "write", l_gpio_write },
160     { "read", l_gpio_read },
161     { NULL, NULL },
162 };
163
164
165 void lua_hw_open(lua_State *L)
166 {
167     sk6812_init(); /* LED pin as an output, LED off */
168     gpio_input(BTN_PIN, GPIO_PULL_UP);
169
170     lua_register(L, "led", l_led);
171     lua_register(L, "button", l_button);
172     lua_register(L, "delay", l_delay);

```

```

173     lua_register(L, "millis", l_millis);
174
175     luaL_newlib(L, gpio_functions);           /* a new table holding the gpio functions ... */
176     lua_setglobal(L, "gpio");                 /* ... stored in the global variable "gpio" */
177 }

```

A `lua_Reg` array lists name and function pairs, ending with two `NULL`s. `luaL_newlib()`, a macro in `luauxlib.h`, creates a table and fills it from the list. `lua_setglobal()` takes the table off the stack and stores it in the global `gpio`, so Lua can call `gpio.write(4, 1)`. Before registering anything, `lua_hw_open()` sets up the LED and gives the button pin its pull-up.

13.5 help(): everything the board adds

app/lua_sys.c help_text, the function list

lines 28–41

```

28     "Functions this board adds to Lua:\n"
29     "   led(r, g, b)           set the RGB LED; each colour 0-255\n"
30     "   button()               true while the button is held\n"
31     "   delay(ms)               wait this many milliseconds\n"
32     "   millis()                milliseconds since start-up\n"
33     "   gpio.output(pin)        make a pin an output (it starts low)\n"
34     "   gpio.input(pin [, pull]) make a pin an input; pull is \"up\", \"down\" or \"none\"\n"
35     "   gpio.write(pin, level)   set an output to 1 or 0\n"
36     "   gpio.read(pin)           1 or 0 (in Lua 0 counts as true, so compare: == 1)\n"
37     "   mem()                    show memory use\n"
38     "   peek(address)            read 32 bits from memory, e.g. hex(peek(0x6000403C))\n"
39     "   hex(value)                write a number in hexadecimal\n"
40     "   crash()                  crash on purpose, to see the crash report\n"
41     "   help()                   this list\n"

```

C joins string literals written next to each other into one string, so `help_text` is one long constant. It is `static const`, so it lives in `.rodata` in flash and takes no RAM. `l_help()` sends it to the console with `fputs()`.

13.6 mem(): the heap and the stack

app/lua_sys.c l_mem()

lines 54–70

```

54 /* mem() */
55 static int l_mem(lua_State *L)
56 {
57     const double KB = 1024.0;
58     struct mallinfo info = mallinfo();           /* newlib's malloc statistics */
59     char *brk = _sbrk(0);                       /* how far malloc has claimed the heap */
60     double lua_kb = lua_gc(L, LUA_GCCOUNT) + lua_gc(L, LUA_GCCOUNTB) / KB;
61
62     printf("heap  %6.1f KB, between the variables (.bss) and the stack\n", (_heap_end - _heap_start) / KB);
63     printf("      %6.1f KB   in use (Lua is using %.1f KB of it)\n", info.uordblks / KB, lua_kb);
64     printf("      %6.1f KB   claimed by malloc but free again\n", info.fordblks / KB);
65     printf("      %6.1f KB   never claimed\n", (_heap_end - brk) / KB);
66     printf("stack %6.1f KB, at most %.1f KB used so far\n", stack_size() / KB, stack_max_used() / KB);
67     if (stack_overflowed())
68         printf("      the stack has OVERFLOWED into the heap: restart the board\n");
69     return 0;
70 }

```

- `mallinfo()` is newlib's report on its heap: `uordblks` is memory in use, `fordblks` is memory `malloc()` claimed earlier that has been freed again.
- `_sbrk(0)` moves the break by nothing and returns where it is, so `_heap_end - brk` is heap never claimed at all (section 11.4).
- `lua_gc()` with `LUA_GCCOUNT` gives the KB Lua itself uses, and `LUA_GCCOUNTB` the bytes left over.

boot/stack_check.c stack checking

lines 13–31

```

13 bool stack_overflowed(void)
14 {
15     return *(uint32_t *)_stack_bottom != STACK_FILL;
16 }
17
18 size_t stack_max_used(void)

```

```

19 {
20     uint32_t *p = (uint32_t *)_stack_bottom;
21
22     /* The stack grows down, so skip up past the words that were never used. */
23     while (p < (uint32_t *)_stack_top && *p == STACK_FILL)
24         p++;
25     return (size_t)((char *)_stack_top - (char *)p);
26 }
27
28 size_t stack_size(void)
29 {
30     return (size_t)(_stack_top - _stack_bottom);
31 }

```

This is the other half of step 2 of `crash0.S`, which painted the whole stack with `STACK_FILL` (section 6.5). `stack_overflowed()` checks whether even the bottom word has changed. `stack_max_used()` starts at the bottom and moves up past every word that still holds the fill value; the first changed word is the deepest the stack has ever gone. This *stack painting* can only tell you afterwards that the stack grew too far, not stop it.

Try this

Type `mem()`, then `t = {} for i = 1, 10000 do t[i] = i end mem()`, then `t = nil collectgarbage() mem()`. Which heap lines changed each time? Then try `load("return " .. string.rep("(", 100) .. "1" .. string.rep(")", 100))` and `mem()`: Lua's parser calls itself for each bracket, and every call takes stack.

13.7 peek() and hex()

app/lua_sys.c l_peek()

lines 72–79

```

72 /* peek(address) -> integer */
73 static int l_peek(lua_State *L)
74 {
75     uintptr_t address = (uintptr_t)(lua_Unsigned)luaL_checkinteger(L, 1);
76
77     lua_pushinteger(L, (lua_Integer)*(volatile uint32_t *)address);
78     return 1;
79 }

```

`peek()` reads 32 bits from any address, exactly as `REG()` does. The cast through `lua_Unsigned` matters because Lua's integers here are 32-bit and signed: addresses from `0x80000000` up are negative numbers in Lua, and the cast turns them back into the right bits. `hex()`, just below, uses `sprintf()` to format a number as `0x` and 8 hexadecimal digits.

What about a bad address?

As the comment in `lua_sys.c` says, nothing protects address 0 on this chip, so `peek(0)` returns whatever is there, without a trap. `crash()`, below, shows a real trap. Other addresses haven't been tried. The exception names in `trap_report.c` include load and store access faults, reading or writing where there is no memory, which a bad address could cause, so be ready to replug the board.

13.8 crash()

app/lua_sys.c l_crash()

lines 96–103

```

96 static int l_crash(lua_State *L)
97 {
98     void (*volatile nowhere)(void) = NULL;    /* volatile: the compiler must really load it and call it */
99
100     (void)L;
101     nowhere();
102     return 0;
103 }

```

`crash()` makes the same mistake as a C program calling a function through a `NULL` pointer: it jumps to address 0. The bytes there are zeros, which are not a valid instruction, so the CPU traps with "illegal instruction". `volatile` stops the compiler noticing that the pointer is always `NULL` and removing the call. Chapter 14 shows what happens next.

13.9 Adding a function of your own

- 1 In `app/lua_hw.c`, copy `l_button()` and rename the copy. Read its arguments with `luaL_checkinteger()`, checked with `luaL_argcheck()` as `channel()` does; push its results and return how many.
- 2 Register it in `lua_hw_open()` with `lua_register()`, and add a line to `help_text` in `app/lua_sys.c`.
- 3 Run `cmake --build build -t flash`, call it at the console, and run `python tools/selftest.py` to check nothing else broke.

Exercise 1a does exactly this with a colour function, `led_hsv()`.

When things go wrong

With no operating system, nobody catches a crash for you. C3U-Metal catches its own and prints a report that says what went wrong and where. This chapter explains that report, the tools for looking inside the board, and how to get a stuck board back.

14.1 Traps: exceptions and interrupts

A **trap** is the CPU abandoning what it was doing and jumping somewhere else. An **exception** is caused by the instruction being run: an illegal instruction, a read from an address with no memory, a breakpoint instruction. An **interrupt** comes from outside, such as a timer or a peripheral asking for attention. When a trap happens, the CPU puts the address of the instruction in the `mepc` CSR and the reason in `mcause`; for some exceptions it also puts the bad address or instruction in `mtval`. Then it jumps to the address in `mtvec`, set by `crt0.S` in step 3. The ROM leaves interrupts switched on, but this program connects no interrupt source to the CPU, so every trap is a crash.

14.2 The trap vector table

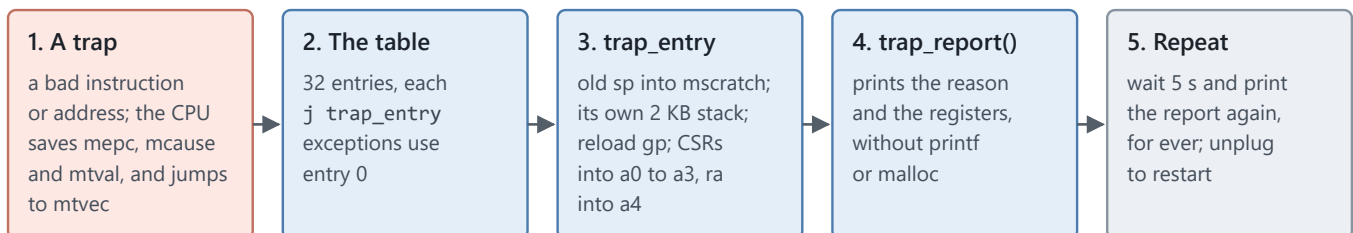
chip/trap.S `trap_vector_table` lines 23–34

```

23     .section .text, "ax"
24
25     .balign 256
26     .global trap_vector_table
27 trap_vector_table:
28     .option push
29     .option norvc                      /* every entry must be exactly 4 bytes */
30     .option norelax
31     .rept 32
32     j    trap_entry
33     .endr
34     .option pop

```

The ESP32-C3 uses `mtvec` in *vector* mode, selected by the `+1` in `crt0.S`: the address points at a table, exceptions go to the first entry, and interrupt n goes to entry n . `.balign 256` starts the table on a multiple of 256, because the CPU ignores the low 8 bits of the table's address. `.rept 32` repeats the jump 32 times. `.option norvc` turns off compressed instructions, so every jump is exactly 4 bytes and entry n is $4 \times n$ bytes from the start. With no interrupt source connected, every entry leads to the same crash handler.



What happens between a crash and the report on your screen.

14.3 The trap handler

chip/trap.S `trap_entry and its stack` lines 42–63

```

42     .balign 4
43 trap_entry:
44     csrw mscratch, sp                /* remember sp as it was at the trap */
45     .option push
46     .option norelax                  /* these must not be made relative to gp */
47     la    sp, trap_stack_top
48     la    gp, __global_pointer$
49     .option pop

```

```

50      csrr a0, mcause          /* why */
51      csrr a1, mepc           /* the instruction that trapped */
52      csrr a2, mtval          /* the bad address or instruction, if any */
53      csrr a3, mscratch       /* sp at the trap */
54      mv  a4, ra              /* where the trapping function was called from (usually) */
55      call trap_report
56 1:    j    1b                /* stop here; a debugger can still look around */
57
58
59      .section .bss
60      .balign 16
61  trap_stack:
62      .skip 2048
63  trap_stack_top:

```

When the handler starts, the program is in an unknown state: the crash might have been caused by a bad `sp` or `gp`, so it trusts neither.

1. `csrr mscratch, sp` keeps the old stack pointer in `mscratch`, a spare CSR meant for this kind of job.
2. `sp` switches to `trap_stack_top`, the top of a 2048-byte stack of its own, reserved in `.bss` by the last lines. `gp` is loaded again, with relaxation off as in `crt0.S`.
3. The CSRs are read into `a0` to `a3` and `ra` is copied into `a4`: by the calling convention, the first five arguments of a C function.
4. `call trap_report` prints the report. `trap_report()` never returns, so the `j 1b` loop after the call is only a last stop.

14.4 The crash report

Near the top of the file, an array called `exception_names` gives the exception codes from the RISC-V specification in plain words, such as code 2, "illegal instruction: the bytes at `mepc` are not an instruction".

app/trap_report.c `print_report()`: the first line

lines 52–63

```

52 static void print_report(uint32_t mcause, uint32_t mepc, uint32_t mtval, uint32_t sp, uint32_t ra)
53 {
54     uint32_t code = mcause & ~MCAUSE_INTERRUPT;
55
56     put_str("\n\n*** CPU trap: ");
57     if (mcause & MCAUSE_INTERRUPT)
58         put_str("an interrupt, but no interrupt source was connected");
59     else if (code < sizeof exception_names / sizeof exception_names[0] && exception_names[code])
60         put_str(exception_names[code]);
61     else
62         put_str("unknown exception");
63     put_str(" ***\n");

```

The report avoids `printf` and `malloc`: as the file says, the crash may have been caused by damaged memory, so it uses as little of the program as it can. `print_report()` checks bit 31 of `mcause` first, because that bit marks an interrupt. Otherwise it looks up the exception's name, checking the code is inside the array and has a name. Then it prints five values with `put_hex()`, which turns a number into hexadecimal by hand, one 4-bit digit at a time, warns if `sp` is outside the stack, and explains how to use `addr2line`.

app/trap_report.c `trap_report()`

lines 79–88

```

79 /* Called by chip/trap.S; never returns. With no terminal open the output is dropped, so keep repeating it. */
80 void trap_report(uint32_t mcause, uint32_t mepc, uint32_t mtval, uint32_t sp, uint32_t ra)
81 {
82     for (;;) {
83         print_report(mcause, mepc, mtval, sp, ra);
84         uint32_t start = cycle_count();
85         while (cycle_count() - start < 5000u * CPU_CYCLES_PER_MS) {
86             ;
87         }
88     }

```

`trap_report()`, the function `trap.S` calls, never returns. The USB driver drops output while no terminal is listening (section 10.4), so it prints the report, waits 5 seconds with the cycle counter, and prints it again, for ever. A terminal opened after the crash still sees the report, and its last line says so.

14.5 Reading a report

Line	What it tells you
*** CPU trap: ... ***	the reason, in words
<code>mcause</code>	the reason as a number: the position in <code>exception_names</code>
<code>mepc</code>	the address of the instruction that trapped
<code>mtval</code>	the bad address or instruction, or 0 if there is none
<code>ra</code>	the return address: usually where the function that trapped was called from
<code>sp</code>	the stack pointer, with a warning if it is outside the stack

Type `crash()` to see one; it repeats every 5 seconds. It says *illegal instruction*, and `mepc` is 0: the CPU tried to run the zeros at address 0. `ra` points back into `l_crash`, the function that made the jump. To turn an address into a function name and source line, or to see the instructions just before `ra`:

Run in a terminal, with an address from the report

```
riscv32-esp-elf-addr2line -f -e build/c3u-metal.elf <address>
riscv32-esp-elf-objdump -d build/c3u-metal.elf --start-address=<ra minus 16> --stop-address=<ra>
```

`addr2line` uses the debugging information in the `.elf` file. `objdump -d` *disassembles*, turning machine code back into assembly, so you can find the `jalr` instruction that made the jump. The program has stopped: unplug the board and plug it back in to restart.

14.6 Looking inside a running board

JTAG is a standard way for a debugging tool to control a chip: stop its CPU, read and write its registers and memory, and set breakpoints. The ESP32-C3's USB port includes a JTAG adapter, so the same cable does it. **OpenOCD** is the program on your computer that speaks JTAG, and **GDB**, the debugger, talks to OpenOCD.

`tools/read_registers.py` uses OpenOCD directly for a quick look. It prints `pc`, `ra`, `sp`, `gp`, the trap CSRs, four hardware registers (the two watchdog configurations, the clock and `GPIO_IN`), and any global variables you name. The heart of it is one list of OpenOCD commands:

`tools/read_registers.py` the OpenOCD commands

lines 79–85

```
79 # One OpenOCD session: halt, print everything with "echo", then resume.
80 # Always resume before shutting down: a CPU left halted stays halted.
81 tcl = ["init", "halt"]
82 tcl += [f"echo \"CPU {name} [reg {name}]\""] for name, _ in CPU_REGISTERS]
83 tcl += [f"echo \"MEM [mdw 0x{addr:08x}]\""] for addr, _ in HARDWARE_REGISTERS]
84 tcl += [f"echo \"MEM [mdw 0x{addr:08x}]\""] for addr in variables.values()]
85 tcl += ["resume", "shutdown"]
```

`halt` stops the CPU, `reg` reads a CPU register, and `mdw` (memory display word) reads 32 bits from an address; `echo` marks each result so Python can find it. `resume` must come before `shutdown`, because a CPU left halted stays halted. Only one program can use JTAG at a time, so stop any debugging session first.

14.7 Debugging in VS Code

Run > Start Debugging (F5) flashes the board, starts OpenOCD, connects GDB and stops at `main()`.

`.vscode/launch.json` has two configurations, chosen at the top of the Run and Debug view. The first, for the C/C++ extension, is the one the project was tested with. It needs `openocd` and `riscv32-esp-elf-gdb` on the PATH, so start VS Code from the ESP-IDF terminal. Its main part:

`.vscode/launch.json` the C/C++ configuration

lines 23–40

```
23     "program": "${workspaceFolder}/build/c3u-metal.elf",
24     "cwd": "${workspaceFolder}",
25     "MIMode": "gdb",
26     "miDebuggerPath": "riscv32-esp-elf-gdb",
27     "miDebuggerServerAddress": "localhost:3333",
28     "preLaunchTask": "Flash, then start OpenOCD",
29     "setupCommands": [
30       { "description": "let GDB read hardware registers at any address", "text": "set mem inaccessible-by-default off" },
31       { "description": "flash is read-only, so GDB uses hardware breakpoints there", "text": "mem 0x42000000 0x42400000 ro" },
32       { "description": "the same flash, as read through the data mapping", "text": "mem 0x3c000000 0x3c400000 ro" },
33       { "description": "the ESP32-C3 has 8 hardware triggers, shared by breakpoints and watchpoints", "text": "set remote hardware-watchpoint-limit 8" },
34     ],
35     "postRemoteConnectCommands": [
36       { "description": "restart the chip and stop before the ROM runs", "text": "monitor reset halt" },
37       { "description": "forget register values from before the reset", "text": "maintenance flush register-cache" },
38     ],
39     { "description": "stop once main() is reached", "text": "thbreak main" },
40     "launchCompleteCommand": "exec-continue"
```

- `preLaunchTask` runs the *Flash* task and then the *OpenOCD* task from `tasks.json`. GDB connects to OpenOCD on port 3333.
- `set mem inaccessible-by-default off` lets GDB read hardware registers at any address.
- The `mem ... ro` lines tell GDB that flash is read-only in both views. GDB can't put an ordinary breakpoint into flash by changing the code there, so it uses the CPU's hardware breakpoints instead. The ESP32-C3 has 8 hardware triggers, shared between breakpoints and watchpoints; a watchpoint stops the program when a variable is read or written.
- After connecting, `monitor reset halt` restarts the chip and stops it before the ROM runs, GDB forgets register values from before the reset, and `thbreak main` sets a temporary breakpoint at `main()`.

Commands typed in the Debug Console need `-exec` in front (section 4.7). OpenOCD keeps running after the session ends. Flashing still works while it runs, but stop it in the terminal panel before the next F5, so that the new OpenOCD can open port 3333.

The second configuration, for Espressif's **ESP-IDF** extension, uses the OpenOCD and GDB that the extension manages, so those needn't be on the PATH. The extension starts OpenOCD itself, with the arguments in `.vscode/settings.json`, and gives up whenever OpenOCD prints an `Error:` line, for example about a breakpoint left over from an earlier session; remove old breakpoints first if it won't start.

14.8 When the stack overflows

app/rep1.c check_stack()

lines 277–287

```
277 /* Nothing stops the stack growing down into the heap, so look after every chunk (and warn once). */
278 static void check_stack(void)
279 {
280     static bool warned;
281
282     if (!warned && stack_overflowed()) {
283         warned = true;
284         fprintf(stderr, "warning: the stack overflowed into the heap, so memory may be damaged. "
285                     "Restart the board.\n");
286     }
287 }
```

The stack is 64 KB, and nothing stops it growing down into the heap. Deeply nested Lua code can do it, because Lua's parser calls itself for each level. The console checks after every chunk and warns once, because after an overflow the heap may be damaged and the only safe thing is to restart. `mem()` reports the same, and the crash report warns if `sp` was outside the stack.

14.9 When nothing appears, or the board is stuck

- **No prompt or banner:** the board prints its prompt only when asked, and output is dropped while no terminal is listening ([section 10.4](#)). Press Enter. `monitor.py` does this when it connects.
- **Monitor opened after a crash:** the report repeats every 5 seconds, so it appears after a moment.
- **Board not found:** try another cable, since many are charge-only. If it still isn't found, or flashing fails, unplug it, **hold the button while plugging it back in**, and flash again. That starts the ROM's download mode ([section 4.2](#)), which works whatever program is on the board.

The lessons in the git history

C3U-Metal wasn't written in one go. Its git history builds it up in five steps, each small enough to understand on its own, and each step has a tag. Reading them in order is a gentle way into the finished code, because the ideas arrive one at a time.

15.1 Exploring the history

Git keeps every version of the project that was committed, and a **tag** is a permanent name for one of them. All but the last two of these commands only read the history:

Command	What it does
<code>git tag</code>	List the tags. <code>pre-cleanup</code> marks the same commit as <code>step-5-lua-repl</code> : the project before it was reorganised into a teaching tool.
<code>git log --oneline</code>	List the commits, newest first.
<code>git show step-1-assembly</code>	Show a step's description and everything it changed.
<code>git show step-1-assembly:main.S</code>	Show one file as it was at that step, without touching your folder.
<code>git diff step-1-assembly step-2-c-runtime</code>	Show exactly what changed between two steps; add <code>-- crt0.S</code> for one file.
<code>git checkout step-2-c-runtime</code>	Switch your folder to that step, to build it.
<code>git checkout main</code>	Back to the latest version.

Before you check out an old step

- Commit your own work first, on a branch of its own ([section 16.2](#)).
- Steps 1 to 5 date from before the CMake build. Their files are all in one folder, with a PowerShell script for Windows: `.\build.ps1` builds and flashes, `.\build.ps1 -NoFlash` only builds. On macOS or Linux, read the steps, or run the `riscv32-esp-elf-gcc` commands from the script by hand.
- Those scripts write into the same `build` folder as CMake, so when you come back to `main`, delete `build` and run `cmake --preset default` again.

15.2 Step 1: assembly only

The first program is two assembly files, a register header and a linker script. `start.S` does the jobs that `crt0.S` and `SystemInit()` do today, and `main.S` lights the LED while the button is held.

start.S at tag *step-1-assembly* _start: the RTC watchdog

lines 9–18

```

9      .section .text.entry, "ax"
10     .global _start
11     _start:
12     /* 1. RTC watchdog: unlock, switch everything off (incl. flash-boot mode), lock */
13     li    t0, RTC_CNTL_WDTWPROTECT
14     li    t1, WDT_WKEY
15     sw    t1, 0(t0)
16     li    t2, RTC_CNTL_WDTCONFIG0
17     sw    zero, 0(t2)
18     sw    zero, 0(t0)
```

Compare this with `SystemInit()` in [section 7.2](#). Without C, each register write takes two or three instructions: `li` loads the register's address and the key into registers, and `sw` stores one at the other. `sw zero, 0(t2)` writes 0 to the configuration and `sw zero, 0(t0)` locks the watchdog again. Further down, `start.S` points `mtvec` at a one-instruction loop, `trap: j trap`, where a debugger can find the CPU, sets `sp`, and ends with `j main`, not `call`, because `main` never returns. There is no `gp` and nothing to copy, because there are no variables.

main.S at tag *step-1-assembly* main: the loop lines 74–84

```

74 loop:
75     li    t0, GPIO_IN
76     lw    t1, 0(t0)
77     andi  t1, t1, 1 << BTN_PIN      /* t1 = 0 while pressed */
78     li    a0, 0                    /* released: LED off */
79     bnez  t1, 1f
80     li    a0, COLOR                /* pressed: LED on  */
81 1:    call send_grb
82     call delay_poll
83     addi  s1, s1, 1
84     j     loop

```

Each pass reads `GPIO_IN`, and `andi` keeps only bit 9, the button. The colour starts as 0; if the bit is set the button isn't pressed and the next line is skipped, otherwise the colour becomes `COLOR`. `send_grb`, already the same bit-banging loop as today's `sk6812_send_grb`, sends it, and `delay_poll` waits 20 ms. `s1` counts passes for a debugger to watch.

c3u-metal.ld at tag *step-1-assembly* sections that aren't supported yet lines 38–49

```

38 /*
39  * Not supported yet: read-only data, initialised data and zeroed data.
40  * Each needs more work (a second flash mapping, or startup code that copies
41  * or clears RAM), so the link fails if any of them are used.
42  */
43 .unsupported (NOLOAD) :
44 {
45     *(.rodata .rodata.* .srodata .srodata.*)
46     *(.data .data.* .sdata .sdata.*)
47     *(.bss .bss.* .sbss .sbss.* COMMON)
48 } > ram
49 ASSERT(SIZEOF(.unsupported) == 0, "c3u-metal.ld: .rodata, .data and .bss are not supported yet")

```

Nothing copies or clears RAM yet, so there can be no `.rodata`, `.data` or `.bss`. Rather than quietly producing a broken program if some code used them, the linker script gathers them into one section and the `ASSERT` stops the link if it isn't empty. Making unsupported things fail loudly is a good habit. The step's `build.ps1` runs one `gcc` command that assembles and links, with no C library and no start-up files.

15.3 Step 2: a C runtime

To write the program in C, it needs the things listed in [section 6.1](#). `start.S` gains a global pointer and the loops that fill `.data` and clear `.bss`; the linker script gains the DROM region and those sections. The program moves into `main.c`, the LED code into `led.S`, and the cycle counter into `cpu.S`. The variables in [section 5.1](#) come from this step.

main.c at tag *step-2-c-runtime* main() lines 63–73

```

63 int main(void)
64 {
65     led_pin_init();
66     button_pin_init();
67
68     for (;;) {
69         send_grb(button_pressed() ? led_color : 0);
70         delay_ms(POLL_MS);
71         loop_count++;
72     }
73 }

```

The same loop as step 1, now in C, reaching the registers through `REG()`. This step also adds `string.c` with `memcpy()`, `memmove()`, `memset()` and `memcmp()`. Even without a C library, GCC may turn a structure copy or a large zero-fill into a call to one of these, so a bare-metal program has to provide them. The build uses `-fno-tree-loop-distribute-`

patterns so the optimiser can't turn the loop inside `memset()` into a call to `memset()`, which would call itself for ever, and links `-lgcc` for helpers such as 64-bit division.

15.4 Step 3: the CMSIS-style split

crt0.S at tag *step-3-cmsis-split* the plan at step 3

lines 1–13

```
1 /*
2  * crt0.S - the C runtime startup for C3U-Metal.
3  *
4  * The ESP32-C3 ROM jumps here (0x42000008) in direct boot mode. Following the
5  * Arm CMSIS pattern, this file only does C-runtime work and calls SystemInit()
6  * (system_esp32c3.c) for everything specific to the chip:
7  *
8  * 1. sp and gp      so C functions can run
9  * 2. mtvec         traps go somewhere a debugger can find
10 * 3. SystemInit()   watchdogs, clock, cycle counter (no global variables yet)
11 * 4. .data, .bss    global variables get their starting values
12 * 5. main()
13 */
```

`start.S` becomes `crt0.S`, and the watchdog and clock code moves into `SystemInit()`, written in C (section 7.1). The order changes too: `sp` and `gp` now come first, because `SystemInit()` is a C function and needs a stack. Compare the list with today's seven steps in section 6.3: painting the stack, the crash-report trap table and copying code into RAM arrive later. Try `git diff --stat step-2-c-runtime step-3-cmsis-split` and `git show step-3-cmsis-split:system_esp32c3.c`.

15.5 Step 4: newlib, the USB console and code in RAM

This step brings in newlib for `printf` and `malloc`, `syscalls.c` to connect it to the hardware, and the USB console driver. The linker script gains the heap and stack areas, newlib's init arrays and `.iram.text`. `crt0.S` learns to copy code into RAM and call `__libc_init_array()`, and `send_grb` moves into RAM. `string.c` is deleted, because newlib has those functions, and the build links `-lc -lm -lgcc` as a group. The test program in `main.c` echoes each key you type, and Enter prints a line that uses `printf` with an integer, a decimal and a pointer from `malloc()`. Read it with `git show step-4-newlib-usb:main.c`.

15.6 Step 5: Lua

The last step adds Lua 5.5.1, unmodified, in `lua/`, built into `liblua.a`. `repl.c` is the console, `lua_hw.c` adds `led`, `button`, `delay` and `millis`, and `uptime.c` keeps the 64-bit time. `main.c` opens the libraries and starts the console, and `syscalls.c` gains the `_fcntl` stub. The program already works much like today's C3U-Metal, with every file in one folder. Try `git diff step-4-newlib-usb step-5-lua-repl -- syscalls.c`.

15.7 After step 5

The commits after the last tag make the project easier to learn from rather than adding new ideas. `git log --oneline step-5-lua-repl..main` lists them, and each message explains what changed and how it was tested.

Commit	What it did
8cc38b2	Sorted the source into folders, one per layer. Pins and CPU speed now come from <code>board.h</code> ; <code>send_grb</code> became <code>sk6812_send_grb</code> ; files that only work at 40 MHz stop the build if <code>CPU_MHZ</code> changes.
cf96e81	Added the GPIO driver and the Lua <code>gpio</code> table; <code>IO_MUX_GPIO(n)</code> replaced hand-written addresses.
29600fd	Added the crash report, stack painting, <code>help()</code> , <code>mem()</code> , <code>peek()</code> , <code>hex()</code> , <code>crash()</code> and <code>read_registers.py</code> .
6e3def3	Replaced <code>build.ps1</code> with CMake and Ninja, added Python tools that work on any operating system, and the VS Code tasks and debug configuration.
f7f1ea6	Documented the project: the README, the diagrams in <code>docs/</code> , a header in each source file, and the licences.

Exercises

The exercises in `docs/exercises` change C3U-Metal to do something new. The first ones change a function or two; the later ones set up hardware that nothing in the project has touched yet. Worked solutions are in `solutions/`.

16.1 The exercises

#	Exercise	What you'll learn	Where you'll work	Background
1	Warm-ups	adding Lua functions in C, reading a crash report	<code>app/luahw.c</code> , <code>chip/trap.S</code>	ch. 13, 14
2	Run at 160 MHz	clock sources, and why timing code cares about CPU speed	<code>chip/system_esp32c3.c</code> , <code>board/board.h</code>	ch. 7, 9
3	A clock that keeps itself	using a hardware timer	<code>drivers/uptime.c</code>	ch. 7
4	A timer interrupt	interrupts, the vector table, saving registers	<code>chip/trap.S</code> , a new driver	ch. 14
5	Keys by interrupt	interrupt-driven input, ring buffers	<code>drivers/usb_serial.c</code>	ch. 10, 12
6	Ctrl-C while typing	the Lua C API, how the console works	<code>app/repl.c</code>	ch. 12
7	Save a script to flash	flash memory, ROM functions, the flash cache	a new driver, <code>app/main.c</code>	ch. 4, 5
8	The LED, with RMT	a peripheral that makes waveforms by itself	<code>drivers/sk6812.*</code>	ch. 9

Exercise 5 needs exercise 4. The others can be done in any order, although 3 is more interesting after 2. The table comes from `docs/exercises/README.md`.

16.2 Working on an exercise

- 1 **Make a branch**, so your changes stay separate from the original: `git switch -c exercise-2`
- 2 **Edit, build and flash**: `cmake --build build -t flash`
- 3 **Check your work**. Each exercise says how. Then run `python tools/selftest.py` to make sure nothing else broke.
- 4 **If you're stuck**, look in Espressif's *ESP32-C3 Technical Reference Manual*, which describes every register, or ESP-IDF's register headers in `components/soc/esp32c3/register/soc/`. After that, look at the worked solution.

If a change stops the board starting, hold its button while plugging it in, then flash a version that works (section 14.9).

16.3 What each exercise asks

- **1. Warm-ups.** Add `led_hsv(h, s, v)` for rainbow colours; write `blink(pin, times, ms)` in Lua for an LED on GPIO4 and a button on GPIO5; then read the `crash()` report with `addr2line` and `objdump`, and add `mstatus` to it. On the C3U it shows `0x00001881`: bits 11 and 12 (MPP) mean machine mode, and bit 7 (MPIE) shows that the ROM left interrupts switched on, although no interrupt source is connected.
- **2. Run at 160 MHz.** Take the CPU clock from the PLL instead of the crystal, set `CPU_MHZ` to 160, and work through every `#error` the build stops at, including measuring the LED's latencies again. A Lua loop that takes about 600 ms at 40 MHz takes about 150 ms.
- **3. A clock that keeps itself.** Make `uptime_us()` read the SYSTIMER, a 52-bit counter that counts 16 000 000 times a second from the crystal whatever the CPU is doing, and make `delay()` use it.
- **4. A timer interrupt.** Make the SYSTIMER interrupt the CPU 1000 times a second through the interrupt matrix, write an interrupt entry in `trap.S` that saves registers and returns with `mret`, keep interrupts off during LED pulses, and add `ticks()`.

- **5. Keys by interrupt.** Receive USB bytes by interrupt into a ring buffer, counting Ctrl-Cs as they arrive. The test: 300 characters pasted during `delay(3000)` all arrive.
- **6. Ctrl-C while typing.** Make Ctrl-C at the `>>` prompt cancel the whole statement. An empty line from Ctrl-C looks just like Enter on an empty line, so `read_line()` needs a way to say "cancelled", passed up through `push_line()` and `multiline()` to `repl_run()`.
- **7. Save a script to flash.** Add `save(code)`, `saved()` and `erase()`, which keep a Lua script in the last 4 KB sector of flash using the ROM's flash functions, called from RAM with the flash cache off. `main.c` runs the script at start-up. A saved script that crashes the board crashes it again at every start-up, and flashing the program again doesn't clear it: `python tools/flash.py --erase` does.
- **8. The LED, with RMT.** Make `sk6812_send_grb()` use the RMT peripheral, which plays a list of 32-bit symbols on the pin with hardware timing, and stop building `sk6812.S`.

16.4 Worked solutions and SOLUTION builds

`solutions/` has a folder for each exercise, such as `02-p11-160mhz`. Each holds complete copies of the files the solution changes or adds, in the same folders as the project, and a `README.md` explaining the solution and how it was tested on the board; files not in the folder come from the project unchanged. `05-usb-rx-interrupt` also contains all of exercise 4's files, because it builds on them. Every solution passes the self-test. Try the exercise before looking. You can build and flash a solution without touching your own files or build folder:

Run in a terminal

```
cmake --preset default -B build-solution -DSOLUTION=04-timer-interrupt
cmake --build build-solution -t flash
git diff --no-index drivers/uptime.c solutions/03-systimer-uptime/drivers/uptime.c
```

`-B build-solution` uses a separate build folder, which `.gitignore` keeps out of git, and `-DSOLUTION=` sets a CMake variable. The last command shows exactly what a solution changed in one file. `CMakeLists.txt` does the rest:

CMakeLists.txt using a solution's files

lines 76–89

```
76 if(SOLUTION)
77     set(SOLUTION_DIR ${CMAKE_CURRENT_SOURCE_DIR}/solutions/${SOLUTION})
78     if(NOT IS_DIRECTORY ${SOLUTION_DIR})
79         message(FATAL_ERROR "There is no solutions/${SOLUTION} folder")
80     endif()
81     message(STATUS "Building the worked solution in solutions/${SOLUTION}")
82     file(GLOB_RECURSE SOLUTION_SOURCES RELATIVE ${SOLUTION_DIR} CONFIGURE_DEPENDS
83          ${SOLUTION_DIR}/*.c ${SOLUTION_DIR}/*.S)
84     list(APPEND SOURCES ${SOLUTION_SOURCES})
85     list(REMOVE_DUPLICATES SOURCES)
86     if(EXISTS ${SOLUTION_DIR}/solution.cmake)
87         include(${SOLUTION_DIR}/solution.cmake)      # lets a solution drop files from SOURCES
88     endif()
89 endif()
```

- `SOLUTION` is a *cache variable*, remembered for that build folder. In a normal build it is empty and none of this happens.
- `file(GLOB_RECURSE ...)` finds every `.c` and `.S` file in the solution folder, as paths such as `drivers/tick.c`, and adds them to `SOURCES`. A copy of an existing file has the same path as the project's entry, so `REMOVE_DUPLICATES` lists it once.
- A `solution.cmake` can take files out. Exercise 8's removes the bit-banging code: `list(REMOVE_ITEM SOURCES drivers/sk6812.S)`.

A function after this, `pick_path()`, turns each listed path into a real file: the solution's copy if it has one, otherwise the project's. The solution's folders come first when looking for headers, and the linker script is chosen the same way.

Register quick reference

Only the registers C3U-Metal's code uses, from `chip/esp32c3-regs.h`. Espressif's ESP32-C3 Technical Reference Manual describes the rest.

A.1 Peripheral registers

Name	Address	What the code uses
RTC_CNTL_WDTCONFIG0	0x60008090	RTC watchdog: bit 31 enable, bit 12 flash-boot mode; <code>SystemInit()</code> writes 0
RTC_CNTL_WDTWPROTECT	0x600080A8	write <code>WDT_WKEY</code> (0x50D83AA1) to unlock, anything else to lock
RTC_CNTL_SWD_CONF	0x600080AC	super watchdog: bit 31 <code>RTC_CNTL_SWD_AUTO_FEED_EN</code> , the chip feeds it itself
RTC_CNTL_SWD_WPROTECT	0x600080B0	write <code>SWD_WKEY</code> (0x8F1D312A) to unlock
TIMG0_WDTCONFIG0	0x6001F048	bit 31 enable, bit 14 flash-boot mode, bit 22 <code>TIMG_WDT_CONF_UPDATE_EN</code> applies changes
TIMG0_WDTWPROTECT	0x6001F064	write <code>WDT_WKEY</code> to unlock
SYSTEM_SYSCLOCK_CONF	0x600C0058	bits 11:10 CPU clock source (0 = crystal); bits 9:0 divider – 1
IO_MUX_GPIO(n)	0x60009004 + 4n	bits 14:12 function (1 = GPIO), bit 9 input enable, bit 8 pull-up, bit 7 pull-down
GPIO_OUT_W1TS, _W1TC	0x60004008, 0x6000400C	write 1 bits: drive those pins high, or low
GPIO_ENABLE_W1TS, _W1TC	0x60004024, 0x60004028	write 1 bits: output driver on, or off
GPIO_IN	0x6000403C	the level of every pin, one bit each; bit 9 is the button
GPIO_FUNC_OUT_SEL_CFG(n)	0x60004554 + 4n	which signal drives GPIO; 128 (<code>SIG_GPIO_OUT_IDX</code>) means <code>GPIO_OUT</code>
USB_SERIAL_EP1	0x60043000	read: the next received byte; write: add a byte to the send buffer
USB_SERIAL_EP1_CONF	0x60043004	bit 0 <code>WR_DONE</code> (write 1: send now), bit 1 <code>TX_FREE</code> (room to send), bit 2 <code>RX_AVAIL</code> (a byte waiting)

A.2 CSRs

CSR	What it holds
<code>mtvec</code>	where traps go; <code>crt0.S</code> sets it to the table address + 1 (vectored mode)
<code>mcause</code>	why the last trap happened; bit 31 set for an interrupt, otherwise an exception code
<code>mepc</code> , <code>mtval</code>	the address of the instruction that trapped; the bad address or instruction, or 0
<code>mscratch</code>	spare; <code>trap.S</code> keeps the old <code>sp</code> here
<code>mstatus</code>	the CPU's status; bits 12:11 (MPP) show the previous privilege mode (exercise 1c)
<code>CSR_PCER</code> 0x7E0, <code>CSR_PCMR</code> 0x7E1	Espressif: 1 in PCER selects counting CPU cycles; 1 in PCMR turns counting on
<code>CSR_PCCR</code> 0x7E2	Espressif: the cycle count, read by <code>cpu.S</code> and <code>sk6812.S</code> ; wraps about every 107 s at 40 MHz

Glossary

Address, address space

The number of a byte, and everything the CPU can reach by number, from `0x00000000` to `0xFFFFFFFF`.

Memory and hardware registers both have addresses.

Assembly language

A readable form of the CPU's own instructions, one per line.

Bare metal

Code that runs with no operating system or framework underneath.

Bit, byte, word

A single 0 or 1; eight bits; on this chip, 32 bits.

Bootloader

A small program that loads and starts the real program. ESP-IDF uses one; C3U-Metal doesn't.

Break

The boundary between claimed and unclaimed heap, moved by `_sbrk()`.

.bss, .data

The sections for global variables that start at zero, and for those with a starting value.

Busy wait

Waiting by running a loop until enough time has passed.

Calling convention

The rules for which registers carry arguments and results, and which a function may change.

CMake, Ninja

Build tools: CMake works out what to build, Ninja runs the compiler.

CMSIS

Arm's common layout for start-up code, which C3U-Metal copies with `SystemInit()`.

Cross-compiler

A compiler that runs on one kind of computer and makes programs for another.

crt0

The C runtime start-up code that runs before `main()`.

CSR

Control and status register: a register inside the CPU, read with `csrr` and written with `csrw`.

Direct boot

The ROM starting a program straight from flash when it finds the magic number.

Download mode

The ROM waiting for esptool, entered by holding GPIO9 low at reset.

DROM, IROM

The data and code views of flash, at `0x3C000000` and `0x42000000`.

ELF file

`c3u-metal.elf`: the program plus addresses, names and debugging information.

Exception, interrupt

A trap caused by the instruction being run; a trap caused by hardware asking for attention.

Flash, flash cache

Memory that keeps the program without power; the part of the chip that loads flash contents for the CPU.

FreeRTOS

A small operating system that runs several tasks; used by ESP-IDF, not by C3U-Metal.

GDB, OpenOCD, JTAG

The debugger; the program that controls the chip; the standard way they do it.

Global pointer (gp)

A register pointing near the small global variables, so one instruction can reach them.

GPIO

General-purpose input/output: a pin the program can drive or read.

Heap

The memory `malloc()` hands out.

Hexadecimal

Base 16, written with `0x`; each digit is four bits.

IO_MUX, pad

The registers that connect each pad to GPIO or a peripheral; the metal contact on the chip that connects to a pin.

Latency

Delay added by the code itself, which `sk6812.S` measures and subtracts.

Linker, linker script

The tool that joins object files and gives everything an address, and the file that tells it where.

Load and run address

Where a section's bytes are stored, and where the program uses them.

Magic number

A value unlikely to occur by accident, used to recognise something.

Memory-mapped I/O

Controlling hardware by reading and writing addresses.

newlib

The C library that comes with the toolchain.

Peripheral

A part of the chip that isn't the CPU, such as GPIO or USB.

Polling

Checking again and again whether something has happened.

Pull-up, pull-down

A weak resistor that holds an input high or low until something drives it.

Read-modify-write

Reading a register, changing some bits, and writing it back.

Register

A small piece of storage: in the CPU, such as `sp`, or in a peripheral, at an address.

REPL

Read, evaluate, print, loop: an interactive prompt.

RISC-V, RV32IMC

The CPU's open instruction set, and the parts of it this chip has.

ROM

Memory with Espressif's start-up code, built into the chip.

Section

A named part of a program, such as `.text`, placed by the linker.

Stack, stack pointer

Scratch memory for function calls, and the register `sp` that marks its top.

Strapping pin

A pin the chip reads at reset to decide how to start. GPIO9 is one.

System call

A function the C library calls to reach the "operating system"; here in `syscalls.c`.

Trap

The CPU abandoning an instruction and jumping to `mtvec`.

volatile

C's way of saying a value can change by itself, so every read and write matters.

W1TS, W1TC

Write 1 to set, write 1 to clear: registers that only change the bits written as 1.

Watchdog

A timer that resets the chip unless it is fed in time.

Troubleshooting

What you see	Why	What to do
<code>riscv32-esp-elf-gcc</code> , <code>cmake</code> or <code>serial</code> not found	The terminal doesn't have ESP-IDF's tools loaded.	Use the ESP-IDF terminal (section 1.5); start VS Code from it with <code>code .</code>
Build errors after moving the folder or building an old lesson step	The build folder remembers old paths and files.	Delete <code>build</code> and run <code>cmake --preset default</code> again.
less than 128 KB left for the heap, or an <code>#error</code> about 40 MHz	Variables leave too little RAM; or <code>CPU_MHZ</code> in <code>board.h</code> changed.	Look for large global arrays you added; put <code>CPU_MHZ</code> back, or see exercise 2.
"M5Stamp C3U not found", or flashing fails	A charge-only cable, or the board isn't answering.	Try another cable, then hold the button while plugging in, and flash again.
"Could not open port", or the port is busy	Another program has the serial port. (OpenOCD uses JTAG, not the serial port, so flashing works while it runs.)	Close the monitor or terminal.
Permission denied on <code>/dev/ttyACM0</code> (Linux)	Your user can't open serial ports.	<code>sudo usermod -aG dialout \$USER</code> , then log out and in.
Nothing appears in the monitor, or no banner	Output is dropped while no terminal is listening.	Press Enter for a prompt.
<code>*** CPU trap ...</code> and the board stops	The crash report (section 14.4); it repeats every 5 s.	Look up <code>mepc</code> and <code>ra</code> with <code>addr2line</code> ; unplug and replug.
The <code>>></code> prompt won't go away	An unfinished statement; Ctrl-C only throws away the line.	Finish the statement, for example with <code>end</code> .
<code>warning: locals do not survive across lines</code>	Each line is its own chunk.	Use global variables at the prompt, or put the code on one line.
<code>if gpio.read(4) then</code> is always true	In Lua, 0 counts as true.	Write <code>if gpio.read(4) == 1 then</code> .
A <code>gpio</code> function refuses a pin, or <code>must</code> be 0-255	The pin belongs to the flash chip or USB, or is the button's or the LED's pin given to <code>gpio.output()</code> or <code>gpio.input()</code> ; or a colour is out of range.	Use another pin; use 0 to 255.
A script saved with exercise 7 crashes the board at every start-up	The saved script traps, and flashing the program again leaves its sector alone.	<code>python tools/flash.py --erase</code> erases the whole flash, then writes the program.
Stack overflow warning, or <code>OVERFLOWED</code> in <code>mem()</code>	More than 64 KB of stack was used.	Restart the board: the heap may be damaged.
The board resets every so often after changing <code>SystemInit()</code>	A watchdog is still running.	Put back steps 1 to 3 of <code>SystemInit()</code> .
Wrong LED colours after changing <code>sk6812.S</code>	The pulse timing changed.	Restore the latency values, keep the code in <code>.iram1</code> , and measure with <code>read_registers.py</code> .
OpenOCD can't find the board (Windows), or <code>read_registers.py</code> can't read the CPU	The JTAG interface needs the WinUSB driver, or another debugger has JTAG.	Install WinUSB with Zadig; stop other debug sessions; replug the board.